1. Понятие и виды программирования. Парадигмы и метафоры программирования. Критерии качества разработки ПО

Давайте начнем с самых основ. **Программирование** — это, по сути, процесс создания инструкций для компьютера. Мы, как программисты, пишем "рецепты" (программы) на специальном языке, которые компьютер затем "исполняет", решая поставленную задачу.

В зависимости от того, *как* мы пишем эти инструкции, выделяют разные **виды (или стили) программирования**. Два основных подхода:

- **Императивное программирование**: Мы пошагово описываем, *как* решить задачу. Мы даем компьютеру последовательность команд: "возьми это", "сложи с тем", "запиши результат сюда". Большинство языков, с которыми вы столкнетесь, включая C++, в своей основе императивные.
- **Декларативное программирование**: Мы описываем, *что* мы хотим получить в итоге, а не как этого достичь. Компьютер сам определяет наилучший способ решения. Примерами могут служить язык SQL для работы с базами данных ("дай мне всех пользователей старше 30") или HTML для разметки веб-страниц ("здесь должен быть заголовок").

Парадигмы программирования

Парадигма — это более широкое понятие, это совокупность идей и концепций, определяющих стиль написания программ.

- **Процедурное программирование**: Это подвид императивного. Программа разбивается на небольшие подзадачи, называемые процедурами или функциями. Каждая функция выполняет одно конкретное действие. Это помогает структурировать код.
- **Объектно-ориентированное программирование (ООП)**: Это фундаментальная парадигма для C++. Здесь мы мыслим в терминах "объектов", которые объединяют в себе и данные (свойства), и действия над этими данными (методы). Например, объект "Автомобиль" может иметь свойства "цвет", "скорость" и методы "ехать()", "тормозить()". Это позволяет создавать сложные, но хорошо организованные системы.
- **Функциональное программирование**: Это подвид декларативного. В его основе лежит математическое понятие функции. Программа рассматривается как вычисление последовательности функций. Ключевые идеи — неизменяемость данных и отсутствие побочных эффектов (функция, получив одни и те же данные, всегда вернет один и тот же результат).
- **Обобщенное (Generic) программирование**: Позволяет писать код, который может работать с различными типами данных, не будучи привязанным к какому-то одному. В C++ это реализуется через шаблоны (templates).

Метафоры программирования

Метафора — это способ думать о программировании, аналогия, которая помогает нам лучше понять сложные концепции. Например, "строительство" — мы проектируем (архитектура), закладываем фундамент (базовые модули), возводим стены (пишем код) и отделываем (тестируем и отлаживаем).

Критерии качества разработки ПО

Хорошая программа — это не просто та, что работает. Она должна соответствовать ряду критериев:

- **Надежность**: Программа должна работать стабильно, без сбоев и ошибок.
- **Корректность**: Программа должна делать именно то, что от нее требуется, и выдавать правильные результаты.
- **Эффективность**: Программа должна использовать ресурсы компьютера (процессорное время, память) экономно.
- **Сопровождаемость (Поддерживаемость)**: Код должен быть написан так, чтобы его было легко читать, понимать и модифицировать в будущем. Это один из важнейших критериев для коммерческой разработки.
- **Переносимость**: Возможность легко адаптировать программу для работы на разных операционных системах или оборудовании.
- **Безопасность**: Программа должна быть защищена от взлома и несанкционированного доступа к данным.

---

2. Основные этапы работы программы. Алгоритм формализация работы ПО.

Основные этапы жизненного цикла ПО:

1. **Анализ требований**: Самый первый и важный этап. Здесь мы должны четко понять, *что* должна делать программа. Какие задачи решать? Какие данные обрабатывать? С кем взаимодействовать? Ошибки на этом этапе обходятся дороже всего.
2. **Проектирование (Архитектура)**: Когда мы знаем "что", нужно решить "как". На этом этапе мы продумываем общую структуру программы. Из каких модулей она будет состоять? Как они будут связаны между собой? Какие технологии и алгоритмы использовать?
3. **Кодирование (Реализация)**: Это непосредственно написание программного кода на выбранном языке программирования. Здесь мы превращаем архитектурные решения в работающие инструкции.
4. **Тестирование**: Очень важный этап, на котором мы проверяем, соответствует ли программа требованиям и не содержит ли она ошибок (багов). Тестирование бывает разным: модульным (проверка отдельных частей), интеграционным (проверка взаимодействия частей), системным (проверка всей программы).
5. **Внедрение (Развертывание)**: Установка программы на компьютеры пользователей или на серверы, чтобы она стала доступна для использования.
6. **Эксплуатация и сопровождение**: После запуска программа требует поддержки: исправления обнаруживаемых ошибок, обновления, добавления нового функционала. Этот этап может длиться годами.

Алгоритм и его формализация

В основе любой программы лежит **алгоритм**.

Алгоритм — это конечная последовательность точных, однозначных шагов (инструкций), которая приводит к решению поставленной задачи.

Ключевые свойства алгоритма:

- **Дискретность**: Алгоритм состоит из отдельных шагов.
- **Детерминированность (Определенность)**: Каждый шаг должен быть строго определен и не допускать двусмысленности.
- **Конечность**: Алгоритм должен завершаться за конечное число шагов.

- **Массовость**: Алгоритм должен быть применим для решения целого класса однотипных задач.
- **Результативность**: Алгоритм должен приводить к какому-то результату.

Формализация работы ПО — это процесс представления работы программы в виде формальной модели, чаще всего в виде алгоритма. Мы можем описать алгоритм разными способами:

- **Словесное описание**: Простое описание шагов на естественном языке.
- **Блок-схема**: Графическое представление алгоритма с помощью стандартных блоков.
- **Псевдокод**: Описание алгоритма на "почти" языке программирования, который использует синтаксические конструкции, но не привязан к конкретному языку.
- **Программный код**: Финальная формализация на конкретном языке программирования.

---

3. Обоснование выбора языка программирования. Характеристика основных языков программирования.

Выбор языка программирования — это важное стратегическое решение, которое зависит от множества факторов. Нет "лучшего" языка на все случаи жизни, есть наиболее подходящий для конкретной задачи.

Критерии выбора языка:

- **Тип задачи**: Для веб-разработки (сайты, веб-приложения) часто выбирают JavaScript, Python, PHP. Для системного программирования (драйверы, операционные системы) — C, C++. Для мобильной разработки — Swift/Objective-C (для iOS) и Kotlin/Java (для Android). Для научных вычислений и анализа данных — Python, R, MATLAB.
- **Производительность**: Если программе требуется максимальная скорость работы (например, в играх, высокочастотном трейдинге), выбирают компилируемые языки вроде C++ или Rust. Для менее критичных к скорости задач подойдут интерпретируемые языки, как Python.
- **Экосистема и библиотеки**: Наличие готовых библиотек и фреймворков может значительно ускорить разработку. У популярных языков (Python, Java, JavaScript) огромные экосистемы.
- **Кроссплатформенность**: Важно ли, чтобы программа работала на Windows, macOS и Linux без изменений? Языки вроде Java (с ее JVM) или Python хорошо справляются с этой задачей. C++ тоже кроссплатформенный, но иногда требует специфичных для платформы доработок.
- **Команда разработчиков**: Какие языки уже знает ваша команда? Нанимать новых специалистов или переучивать старых — это временные и денежные затраты.
- **Поддержка и сообщество**: Большое и активное сообщество означает, что вы легко найдете ответы на вопросы, учебные материалы и готовые решения.

Характеристика основных языков

- **C++**:
  - **Парадигмы**: Процедурная, объектно-ориентированная, обобщенная.
  - **Особенности**: Высокая производительность, прямой доступ к памяти (через указатели), строгая статическая типизация. Это мощный, но сложный язык.
  - **Применение**: Системное ПО, высокопроизводительные приложения, игровые движки (Unreal Engine), финансовые системы, встраиваемые системы.

- **Python**:
  - **Парадигмы**: Объектно-ориентированная, процедурная, функциональная.
  - **Особенности**: Простой и лаконичный синтаксис, динамическая типизация, огромная стандартная библиотека. Интерпретируемый (обычно медленнее C++).
  - **Применение**: Веб-разработка (Django, Flask), анализ данных, машинное обучение (TensorFlow, PyTorch), научные вычисления, автоматизация.
- **Java**:
  - **Парадигмы**: Строго объектно-ориентированная.
  - **Особенности**: Принцип "написано однажды, работает везде" (WORA) благодаря виртуальной машине Java (JVM). Автоматическое управление памятью. Строгая статическая типизация.
  - **Применение**: Крупные корпоративные системы, веб-приложения (backend), Android-разработка, работа с большими данными (Big Data).
- **JavaScript**:
  - **Парадигмы**: Прототипно-ориентированная, функциональная, императивная.
  - **Особенности**: Язык веба. Выполняется в браузере клиента. Динамическая типизация. Огромная экосистема (Node.js для backend, React/Angular/Vue для frontend).
  - **Применение**: Frontend (интерактивность на сайтах), backend (Node.js), мобильные приложения (React Native), десктопные приложения (Electron).
- **C#**:
  - **Парадигмы**: Объектно-ориентированная, компонентно-ориентированная.
  - **Особенности**: Разработан Microsoft. Похож на Java. Работает на платформе .NET. Строгая статическая типизация, автоматическое управление памятью.
  - **Применение**: Разработка под Windows, веб-приложения (ASP.NET), игровые движки (Unity).

---

4. Процесс формирования исполняемой программы. Состав языка программирования.

Когда мы пишем код, мы создаем текстовый файл, понятный человеку. Но компьютер понимает только машинный код — последовательности нулей и единиц. Процесс превращения нашего текста в исполняемую программу состоит из нескольких этапов.

Процесс компиляции и сборки:

1. **Препроцессинг**: Препроцессор — это первая программа, которая обрабатывает ваш исходный код. Он выполняет директивы, начинающиеся с символа `#`. Например, `#include <iostream>` говорит препроцессору взять содержимое файла `iostream` и вставить его прямо в ваш код. `#define PI 3.14` заменяет все вхождения `PI` на `3.14`.
2. **Компиляция**: Компилятор берет результат работы препроцессора и переводит его в **ассемблерный код** — низкоуровневый, но все еще читаемый человеком язык, близкий к машинным инструкциям. Затем этот ассемблерный код переводится в **объектный код** (файлы с расширениями `.o` или `.obj`). Это уже почти машинный код, но он еще не является готовой программой. В нем могут быть ссылки на функции, код которых находится в других файлах или библиотеках.
3. **Линковка (Сборка)**: Линковщик (или компоновщик) берет один или несколько объектных файлов, добавляет к ним код из стандартных библиотек (например, код для функции `std::cout`) и собирает все это в единый **исполняемый файл** (`.exe` в Windows, без расширения в Linux/macOS).

Состав языка программирования

Любой язык, будь то человеческий или компьютерный, имеет свою структуру. Язык программирования состоит из:

- **Алфавит**: Набор символов, которые можно использовать в языке. Для C++ это латинские буквы, цифры, знаки препинания, специальные символы (`+`, `-`, `*`, `/`, `{`, `}` и т.д.).
- **Лексика (Лексемы)**: Минимальные единицы языка, имеющие смысл. Подобно словам в русском языке. К лексемам относятся:
  - **Ключевые (зарезервированные) слова**: `int`, `double`, `if`, `else`, `for`, `while`, `class`. Их нельзя использовать в качестве имен переменных.
  - **Идентификаторы**: Имена, которые мы даем переменным, функциям, классам (например, `my_variable`, `calculateSum`).
  - **Константы (литералы)**: Фиксированные значения, такие как `123`, `3.14`, `"Hello, World!"`.
  - **Знаки операций**: `+`, `-`, `=`, `*`, `==`.
  - **Разделители**: `(`, `)`, `{`, `}`, `;`, `,`.
- **Синтаксис**: Набор правил, по которым из лексем строятся правильные конструкции языка (выражения, операторы, функции). Если вы нарушите синтаксис (например, забудете точку с запятой), компилятор выдаст ошибку.
- **Семантика**: Смысл, который несут синтаксические конструкции. Код `a = b + c;` синтаксически корректен. Его семантика заключается в том, чтобы сложить значения переменных `b` и `c` и присвоить результат переменной `a`.

Пример кода (Демонстрация состава ++языка)

Давайте разберем простую программу с точки зрения состава языка.

C++
```
#include <iostream>

// Эта программа вычисляет площадь круга

int main() { // 'int' и 'main' - ключевое слово и идентификатор
    const double PI = 3.14; // 'const', 'double' - ключевые слова; 'PI' - идентификатор; '=' - операция; '3.14' - константа; ';' - разделитель
    double radius = 5.0;    // 'double' - ключевое; 'radius' - идентификатор; '=' - операция; '5.0' - константа; ';' - разделитель

    double area = PI * radius * radius; // 'area' - идентификатор; '=' и '*' - операции

    // 'std::cout' и 'std::endl' - идентификаторы из пространства имен std
    // '<<' - операция вывода
    // "Площадь круга: " - строковая константа (литерал)
    std::cout << "Площадь круга: " << area << std::endl;

    return 0; // 'return' - ключевое слово; '0' - константа
}
```

- **Алфавит**: все использованные символы (`i`, `n`, `t`, `(`, `)`, `{`, `}` и т.д.).
- **Лексемы**: `int`, `main`, `(`, `)`, `{`, `const`, `double`, `PI`, `=`, `3.14`, `;`, `radius`, `area`, `*`, `std`, `::`, `cout`, `<<`, `"Площадь круга: "`, `endl`, `return`, `0`, `}`.
- **Синтаксис**: Каждая строка заканчивается `;`. `main` имеет круглые скобки и тело в фигурных скобках. Переменные объявляются с указанием типа.
- **Семантика**: Программа объявляет константу `PI` и переменную `radius`, вычисляет их произведение для нахождения площади и выводит результат на экран.

5. Типы данных в языке C++. Размерность адресация. Диапазоны типов по стандарту. Константы в языке C++. Целочисленный тип данных.

В C++ каждая переменная должна иметь **тип**. Тип данных говорит компилятору, какого рода информацию будет хранить переменная (целое число, дробное, символ и т.д.) и сколько места в памяти для этого нужно выделить. C++ является языком со **строгой статической типизацией**. Это значит, что тип переменной определяется на этапе компиляции и не может быть изменен в процессе выполнения программы.

Основные встроенные типы данных:

- **Целочисленные**:
  - `int`: Основной тип для целых чисел.
  - `short` (или `short int`): Для "коротких" целых. Занимает меньше памяти, чем `int`.
  - `long` (или `long int`): Для "длинных" целых.
  - `long long` (или `long long int`): Для очень больших целых чисел.
  - `char`: Технически, это тоже целочисленный тип, но используется для хранения одного символа (например, 'a', 'Z', '?').
- **Типы с плавающей точкой (вещественные)**:
  - `float`: Числа с одинарной точностью.
  - `double`: Числа с двойной точностью (более точные, чем `float`, и используются по умолчанию для дробных чисел).
  - `long double`: Числа с повышенной точностью.
- **Логический тип**:
  - `bool`: Может принимать только два значения: `true` (истина) или `false` (ложь).
- **Пустой тип**:
  - `void`: Означает "отсутствие типа". Используется для функций, которые не возвращают значение, или для указателей на "что угодно".

Размерность и диапазоны

Размер типа (сколько байт он занимает в памяти) зависит от архитектуры компьютера и компилятора. Однако стандарт C++ гарантирует определенные минимальные соотношения, например, `sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)`.

Узнать размер типа на вашей системе можно с помощью оператора `sizeof()`.

Примерные диапазоны (на типичной 64-битной системе):

Тип данных	Размер (байт)	Диапазон значений	Эквивалентные названия типа
char	1	−128...+127	signed char
int	2/4	зависит от системы	signed, signed int
unsigned char	1	0...255	нет
unsigned int	2/4	зависит от системы	unsigned
short int	2	−32768...32767	short, signed short int
unsigned short	2	0...65535	unsigned short int
long int	4	−2147483648...2147483647	long, signed long int
unsigned long int	4	0...4294967295	unsigned long
float	4	±(3.4E−38...3.4E+38)	нет
double	8	±(1.7E−308...1.7E+308)	нет
long double	10	±(3.4E−4932...1.1E+4932)	нет

Модификаторы `signed` **и** `unsigned`: Целочисленные типы по умолчанию являются `signed` (знаковыми), то есть могут хранить и отрицательные, и положительные числа. Модификатор `unsigned` (беззнаковый) указывает, что переменная будет хранить только положительные числа и ноль. Это позволяет вдвое увеличить верхнюю границу диапазона.

Константы в C++

Константа — это переменная, значение которой нельзя изменить после инициализации.

Есть два основных способа объявить константу:

1. **Ключевое слово** `const`: Наиболее предпочтительный и современный способ в C++.

   C++
   ```
   const int MAX_USERS = 100;
   const double PI = 3.14159;
   ```

2. **Директива препроцессора** `#define`: Старый способ, унаследованный от языка C. Он просто выполняет текстовую подстановку перед компиляцией.

   C++
   ```
   #define SCREEN_WIDTH 1920
   ```

   Использование `const` безопаснее, так как компилятор знает тип константы и может выполнять проверки.

---

6. Структура простейшей программы на языке C++. Переменные, операции, выражения, преобразования базовых типов.

Давайте рассмотрим "скелет" самой базовой программы на C++ и разберем ее составные части.

Структура простейшей программы

C++
```
// 1. Директивы препроцессора (подключение библиотек)
#include <iostream>

// 2. Главная функция - точка входа в программу
int main()
{ // 3. Начало тела функции
    // 4. Инструкции (операторы) программы
    std::cout << "Hello, World!" << std::endl;

    // 5. Возвращаемое значение
    return 0;
} // 6. Конец тела функции
```

1. `#include <iostream>`: Подключает стандартную библиотеку ввода-вывода. Она нужна, чтобы мы могли использовать `std::cout` для вывода текста в консоль.
2. `int main()`: Любая программа на C++ должна иметь функцию `main`. Это та точка, с которой операционная система начинает выполнение вашего кода. `int` перед `main` означает, что функция по завершении вернет операционной системе целое число (код завершения).
3. `{ }`: Фигурные скобки определяют начало и конец блока кода, в данном случае — тела функции `main`.
4. **Инструкции**: Внутри фигурных скобок располагаются команды, которые должна выполнить программа. Каждая инструкция в C++ должна заканчиваться точкой с запятой (`;`).
5. `return 0;`: Эта инструкция завершает функцию `main` и возвращает операционной системе код `0`. По соглашению, код `0` означает, что программа завершилась успешно.

Переменные, операции, выражения

- **Переменная**: Это именованная область в памяти, предназначенная для хранения данных определенного типа. Прежде чем использовать переменную, ее нужно **объявить**, указав ее тип и имя.

C++
```
int age; // Объявление переменной
age = 25; // Инициализация (присваивание первого значения)

double price = 99.99; // Объявление и инициализация в одной строке
```

- **Операция**: Это символ, который выполняет определенное действие над одним или несколькими операндами (переменными или значениями).
  - Арифметические: `+` (сложение), `-` (вычитание), `*` (умножение), `/` (деление), `%` (остаток от деления).
  - Присваивания: `=` (присвоить значение).
- **Выражение**: Это комбинация переменных, констант и операций, которая вычисляется и дает некий результат.
  - `5` - простое выражение.
  - `age + 1` - выражение, результатом будет возраст, увеличенный на 1.

- `price * 2` - выражение.
- `x = a + b * c` - `a + b * c` это выражение, результат которого присваивается `x`. Вся строка является **оператором присваивания**.

Преобразования базовых типов

Иногда C++ автоматически преобразует один тип данных в другой. Это называется **неявным преобразованием**.

C++
```
int a = 3.14; // double 3.14 будет неявно преобразован в int 3. Дробная часть отбрасывается!
double b = 5; // int 5 будет неявно преобразован в double 5.0
```

Неявные преобразования работают, когда нет риска потери данных (как `int` в `double`). Когда риск есть (как `double` в `int`), компилятор может выдать предупреждение.

Также существует **явное преобразование**, когда мы сами указываем компилятору, какой тип хотим получить.

C++
```
int total_sum = 100;
int items_count = 8;

// Неправильно! 100 / 8 = 12 (целочисленное деление)
double average_wrong = total_sum / items_count;

// Правильно! Явно преобразуем одну из переменных к double
double average_correct = static_cast<double>(total_sum) / items_count;
```

Использование `static_cast<new_type>(value)` — это современный и безопасный способ явного преобразования типов в C++.

---

7. Область видимости переменных. Характеристика классов памяти. Пространства имён.

Не все переменные в программе "видны" из любого ее места. Концепция, которая это регулирует, называется **областью видимости**.

Область видимости (scope) — это часть программы, в которой имя переменной является уникальным и может быть использовано.

В C++ в основном выделяют следующие области видимости:

- **Локальная область видимости (область видимости блока)**: Переменная, объявленная внутри блока кода (например, внутри функции или просто внутри фигурных скобок `{}`), видна только от точки ее объявления и до конца этого блока. Такие переменные называются **локальными**.
- **Глобальная область видимости**: Переменная, объявленная вне всех функций, является **глобальной**. Она видна из любой точки программы после ее объявления. Использовать глобальные переменные следует с большой осторожностью, так как это может затруднить понимание и отладку кода.

Классы памяти

Класс памяти определяет **время жизни** и **область видимости** переменной.

- `auto` **(автоматические)**: Это класс памяти по умолчанию для всех локальных переменных. Переменная создается при входе в ее область видимости (блок) и уничтожается при выходе из него. Память для нее выделяется на **стеке**.
- `static` **(статические)**:
  - Если `static` применить к **локальной переменной**, она будет существовать на протяжении всего времени работы программы, а не уничтожаться при выходе из блока. Ее значение сохраняется между вызовами функции. Память для нее выделяется в специальном сегменте данных.
  - Если `static` применить к **глобальной переменной или функции**, это ограничит ее видимость только текущим файлом (единицей трансляции).
- `extern` **(внешние)**: Используется для того, чтобы сообщить компилятору, что переменная объявлена в другом файле. Это позволяет использовать одну глобальную переменную в нескольких файлах проекта.
- `register`: Устаревший класс памяти. Это была "просьба" к компилятору разместить переменную в регистре процессора для максимально быстрого доступа. Современные компиляторы игнорируют это ключевое слово и сами решают, что размещать в регистрах.

Пространства имён (Namespaces)

Когда проекты становятся большими, или когда вы используете несколько библиотек, может возникнуть **конфликт имен**. Например, в вашей программе есть функция `log()`, и в подключенной библиотеке тоже есть функция `log()`. Компилятор не поймет, какую из них вызывать.

Пространство имён — это механизм, позволяющий сгруппировать идентификаторы (имена переменных, функций, классов) под общим именем, чтобы избежать таких конфликтов.

Стандартная библиотека C++ помещена в пространство имен `std`. Именно поэтому мы пишем `std::cout` и `std::endl`. Мы явно указываем, что хотим использовать `cout` из пространства имен `std`.

Чтобы не писать `std::` каждый раз, можно использовать директиву `using`:

- `using std::cout;` — делает видимым только `cout` из `std`.
- `using namespace std;` — делает видимыми *все* имена из `std`. **В учебных примерах это допустимо, но в больших проектах (особенно в заголовочных файлах) это считается плохой практикой**, так как может привести к тем самым конфликтам имен, для борьбы с которыми пространства имен и были созданы.

---

8. Основные правила использования переменных. Понятие время жизни переменной.

Правильное обращение с переменными — залог написания чистого, понятного и работающего кода. Давайте сформулируем несколько ключевых правил.

Основные правила использования переменных:

1. **Давайте осмысленные имена**: Имя переменной должно отражать ее суть. Вместо `int x;` лучше написать `int user_count;` или `int current_temperature;`. Это делает код самодокументируемым.
2. **Объявляйте переменную как можно ближе к месту ее первого использования**: Не стоит объявлять все переменные в начале функции. Это улучшает читаемость и помогает ограничить область видимости.
3. **Инициализируйте переменные при объявлении**: Неинициализированная переменная содержит "мусор" — случайное значение, которое было в этом участке памяти. Попытка использовать такую переменную ведет к непредсказуемому поведению программы.

   C++
   ```
   // Плохо
   int value;
   // ... много кода ...
   value = 10;

   // Хорошо
   int value = 10;
   ```

4. **Используйте константы для значений, которые не должны меняться**: Если у вас есть значение, которое является постоянным (например, число Пи, максимальное количество подключений), объявите его как `const`. Это защитит от случайного изменения и сделает код понятнее.
5. **Минимизируйте использование глобальных переменных**: Глобальные переменные могут быть изменены из любой части программы, что делает отладку крайне сложной. Старайтесь передавать данные в функции через параметры.
6. **Выбирайте подходящий тип данных**: Не используйте `double` для хранения количества студентов, для этого достаточно `int` или `unsigned int`. Использование правильного типа экономит память и предотвращает ошибки.

   Время жизни переменной (Lifetime)

   Время жизни — это интервал времени выполнения программы, в течение которого переменная существует в памяти. Это понятие тесно связано с классом памяти.

- **Автоматические переменные (локальные)**: Их время жизни ограничено блоком, в котором они объявлены. Они "рождаются" в момент объявления и "умирают" (память освобождается) при выходе из блока (когда выполнение доходит до закрывающей фигурной скобки `}`).
- **Статические и глобальные переменные**: Их время жизни равно времени работы всей программы. Они "рождаются" до вызова функции `main` и "умирают" после ее завершения.

  Важно не путать **время жизни** и **область видимости**.

- **Область видимости**: *Где* в коде мы можем обратиться к переменной по имени.
- **Время жизни**: *Когда* во время выполнения программы переменная существует в памяти.

  У статической локальной переменной **область видимости** — локальная (внутри ее блока), но **время жизни** — глобальное (на все время работы программы).

---

9. Основные операции C++: унарные, бинарные, тернарные, поразрядные операции. Приоритет операций. Особенности операций инкремента и декремента.

Операции в C++ можно классифицировать по количеству операндов, которые они принимают.

- **Унарные операции**: Применяются к одному операнду.
  - `-a`: Унарный минус (меняет знак).
  - `+a`: Унарный плюс (ничего не меняет, используется для симметрии).
  - `++a` (префиксный инкремент), `a++` (постфиксный инкремент): Увеличивает на 1.
  - `--a` (префиксный декремент), `a--` (постфиксный декремент): Уменьшает на 1.
  - `!` (логическое НЕ): Инвертирует логическое значение (`!true` есть `false`).
  - `&a`: Взятие адреса переменной.
  - `*p`: Разыменование указателя.
  - `sizeof(a)`: Определение размера в байтах.
- **Бинарные операции**: Применяются к двум операндам.
  - Арифметические: `a + b`, `a - b`, `a * b`, `a / b`, `a % b`.
  - Присваивания: `a = b`, `a += b`, `a -= b` и т.д.
  - Сравнения: `a == b`, `a != b`, `a > b`, `a < b`, `a >= b`, `a <= b`.
  - Логические: `a && b` (логическое И), `a || b` (логическое ИЛИ).
- **Тернарная операция**: Единственная операция в C++, которая принимает три операнда.
  - `условие ? выражение1 : выражение2`
  - Работает так: если `условие` истинно (`true`), то вычисляется и возвращается `выражение1`, в противном случае — `выражение2`. Это удобная замена для простых конструкций `if-else`.

Поразрядные (побитовые) операции

Эти операции работают с отдельными битами в целочисленных типах данных.

- `&` (побитовое И): Устанавливает бит в 1, только если оба соответствующих бита операндов равны 1.
- `|` (побитовое ИЛИ): Устанавливает бит в 1, если хотя бы один из соответствующих битов операндов равен 1.
- `^` (побитовое исключающее ИЛИ / XOR): Устанавливает бит в 1, если соответствующие биты операндов различны.
- `~` (побитовое НЕ / инверсия): Инвертирует все биты операнда (0 становится 1, 1 становится 0).
- `<<` (сдвиг влево): Сдвигает биты влево на указанное количество позиций (эквивалентно умножению на степень двойки).
- `>>` (сдвиг вправо): Сдвигает биты вправо (эквивалентно целочисленному делению на степень двойки).

Таблица приоритетов операторов C++

Приоритет	Категория / Операторы	Ассоциативность	Комментарий
1	: :	—	Оператор области видимости

Приоритет	Категория / Операторы	Ассоциативность	Комментарий
2	++ (постфикс), -- (постфикс), () (вызов функции), [], . , ->, typeid, const_cast, dynamic_cast, reinterpret_cast, static_cast	**слева → направо**	Постфиксные операции, обращение к элементам, вызов функций
3	++ (префикс), -- (префикс), + (унарный), - (унарный), !, ~, * (разыменование), & (взятие адреса), sizeof, alignof, new, delete, co_await	**справа ← налево**	Унарные операции
4	.*, ->*	**слева → направо**	Доступ к члену через указатель на член
5	*, /, %	**слева → направо**	Умножение, деление, остаток
6	+, -	**слева → направо**	Сложение, вычитание
7	<<, >>	**слева → направо**	Побитовые сдвиги
8	<, <=, >, >=	**слева → направо**	Сравнение
9	==, !=	**слева → направо**	Проверка равенства
10	&	**слева → направо**	Побитовое И
11	^	**слева → направо**	Побитовое XOR
12	`	`	**слева → направо**
13	&&	**слева → направо**	Логическое И (с коротким замыканием)
14	`		`
15	?: (тернарный оператор)	**справа ← налево**	Условный оператор
16	=, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, `	=`	**справа ← налево**
17	throw	**справа ← налево**	Бросание исключений
18	,	**слева → направо**	Запятая (оператор последовательности)

Краткое правило запоминания

- **Сначала** — обращения к членам, вызовы функций, индексация (. -> () []).
- **Потом** — унарные операции (++, --, !, ~, +, -, *, &).
- **Далее** — арифметика (*, /, %, +, -).
- **Потом** — сдвиги (<<, >>), сравнения (<, > …), равенства (==, !=).
- **Затем** — побитовые (&, ^, |) и логические (&&, ||).
- **В конце** — тернарный ?:, присваивания =, и запятая ,.

Особенности инкремента и декремента

Это очень важный момент, в котором часто путаются.

- **Префиксная форма (++a, --a)**: Сначала значение переменной **изменяется**, а затем **результат** (уже измененное значение) используется в выражении.
- **Постфиксная форма (a++, a--)**: Сначала **старое значение** переменной используется в выражении, и только **потом** значение самой переменной изменяется.

10. Вводные замечания о средствах ввода-вывода, особенности вывода кириллицы в консольное окно.

Для взаимодействия с пользователем программа должна уметь принимать данные (ввод) и отображать результаты (вывод). В C++ для этого используется библиотека `<iostream>`.

- `std::cout` **(console output)**: Стандартный поток вывода. Используется для вывода данных в консоль. Работает в паре с оператором `<<` (оператор вставки в поток).
- `std::cin` **(console input)**: Стандартный поток ввода. Используется для считывания данных из консоли (обычно с клавиатуры). Работает в паре с оператором `>>` (оператор извлечения из потока).
- `std::endl` **(end line)**: Это манипулятор потока. Он делает две вещи: вставляет в поток символ новой строки (`\n`) и **сбрасывает буфер** потока вывода (то есть немедленно выводит все, что накопилось, на экран).

C++
```
int age;
std::string name;

std::cout << "Введите ваше имя: ";
std::cin >> name; // Считывает одно слово до пробела

std::cout << "Введите ваш возраст: ";
std::cin >> age;

std::cout << "Привет, " << name << "! Вам " << age << " лет." <<
std::endl;
```

Особенности вывода кириллицы в консоль

Это одна из распространенных проблем, с которой сталкиваются начинающие C++ программисты в Windows. Проблема связана с **кодировками**.

- Ваш исходный файл `.cpp` сохранен в одной кодировке (часто это UTF-8).
- Консоль Windows по умолчанию использует другую кодировку (обычно CP866 или CP1251).

Когда вы пытаетесь вывести строку "Привет", компилятор записывает в программу байты, соответствующие этой строке в UTF-8. Консоль же, получив эти байты, пытается интерпретировать их по своим правилам (CP866) и в результате показывает "кракозябры".

Решения (для Windows):

1. **Настройка локали**: Самый правильный и кроссплатформенный способ. Нужно в начале программы установить русскую локаль для всех категорий. Это говорит потокам ввода-вывода, как правильно обрабатывать символы.
2. **Использование функций WinAPI**: Менее переносимый способ, привязанный к Windows. Можно использовать функции `SetConsoleCP` и `SetConsoleOutputCP` для смены кодировки консоли прямо из программы.

Для других операционных систем (Linux, macOS) эта проблема обычно не возникает, так как они по умолчанию используют UTF-8.

Важное замечание: Для того чтобы этот код сработал, ваш **файл с исходным кодом** также должен быть сохранен в кодировке **UTF-8**. Убедитесь, что ваша среда разработки (IDE) настроена соответствующим образом.

---

11. Условный оператор: понятие, синтаксис, блок-схема. Тернарные операции. Примеры69.

Понятие и назначение

Представьте, что программа — это дорога, по которой движется исполнитель. В какой-то момент возникает развилка. Куда пойти — налево или направо? Решение зависит от какого-то условия. **Условный оператор** — это и есть та самая "развилка" в коде. Он позволяет программе выполнять разные блоки кода в зависимости от того, истинно (`true`) или ложно (`false`) определённое условие. Это фундаментальный инструмент для создания нелинейных, "думающих" программ.

Синтаксис `if`, `else if`, `else`

Существует три основные формы условного оператора:

1. **Простой `if`**: Выполняет блок кода, только если условие истинно.

   C++

   ```
   if (условие) {
       // Код, который выполнится, если условие истинно
   }
   ```

2. **`if-else`**: Выбирает один из двух блоков кода.

   C++

   ```
   if (условие) {
       // Код, который выполнится, если условие истинно
   } else {
       // Код, который выполнится, если условие ложно
   }
   ```

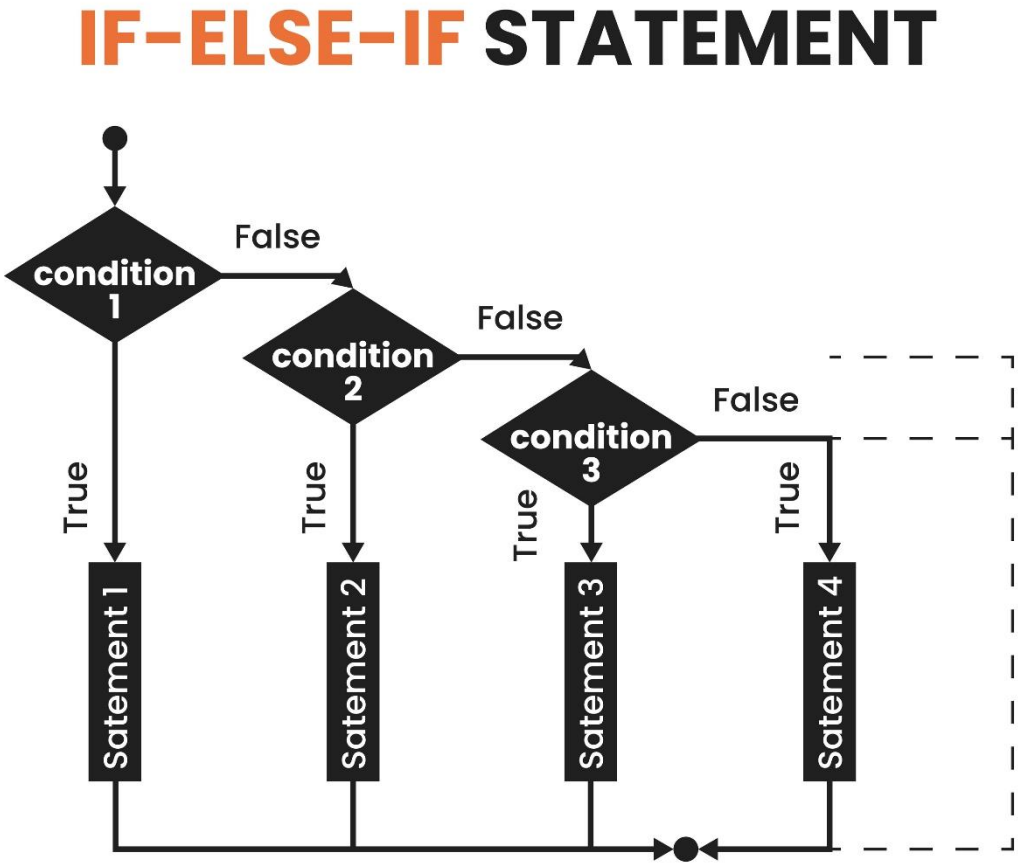
3. **`if-else if-else`**: Цепочка проверок. Позволяет выбрать один из множества путей.

   C++

   ```
   if (условие_1) {
       // Блок 1
   } else if (условие_2) {
       // Блок 2
   } else {
       // Блок 3 (выполнится, если ни одно из условий выше не истинно)
   }
   ```

Блок-схема `if-else`

Визуально логику оператора `if-else` можно представить так:



Программа доходит до ромба (условия). Если ответ "Да" (true), она идёт по одной ветке. Если "Нет" (false) — по другой. После выполнения одной из веток программа продолжает свой путь.

Тернарная операция

Тернарный оператор (`?:`) — это сокращённая, компактная форма записи `if-else`, которая возвращает одно из двух значений. Его часто используют для присваивания значения переменной.

Синтаксис: `условие ? выражение_если_true : выражение_если_false;`

Он читается так: "Если `условие` истинно, вернуть `выражение_если_true`, иначе вернуть `выражение_если_false`".

Примеры кода

C++

```
#include <iostream>
#include <string>
```

```
int main() {
    int score = 85;

    // 1. Пример с if-else if-else
    std::cout << "--- Пример с if-else if-else ---" << std::endl;
    if (score > 90) {
        std::cout << "Оценка: A (Отлично)" << std::endl;
    } else if (score > 75) {
        std::cout << "Оценка: B (Хорошо)" << std::endl; // Этот блок
выполнится
    } else if (score > 60) {
        std::cout << "Оценка: C (Удовлетворительно)" << std::endl;
    } else {
        std::cout << "Оценка: F (Неудовлетворительно)" << std::endl;
    }

    // 2. Пример с тернарным оператором
    std::cout << "\n--- Пример с тернарным оператором ---" << std::endl;
    int age = 20;
    // Определяем, совершеннолетний ли пользователь
    std::string status = (age >= 18) ? "Совершеннолетний" :
"Несовершеннолетний";

    std::cout << "Ваш статус: " << status << std::endl;

    return 0;
}
```

---

12. Синтаксис оператора `switch`. Примеры.

Понятие и назначение

Оператор `switch` (переключатель) — это ещё один способ организовать ветвление в программе. Он является удобной альтернативой длинным конструкциям `if-else if-else`, когда вам нужно сравнить значение **одной** переменной с несколькими возможными константами.

`switch` работает только с целочисленными типами (`int`, `char`, `enum`) и не может проверять диапазоны (например, `x > 10`), в отличие от `if`.

Синтаксис

C++

```
switch (переменная_или_выражение) {
    case константа_1:
        // Код для случая, когда переменная == константа_1
        break; // Обязательный выход из switch

    case константа_2:
        // Код для случая, когда переменная == константа_2
        break;

    case константа_3:
    case константа_4:
        // "Проваливание": этот код выполнится, если переменная
        // равна константе_3 ИЛИ константе_4
        break;
```

```
    default:
        // Код, который выполнится, если ни один case не подошел
        // (аналог последнего else)
        break;
}
```

Ключевые моменты:

- **`case`**: Сравнивает значение в скобках `switch` со своей константой.
- **`break`**: **Критически важный оператор!** Он немедленно прекращает выполнение `switch`. Если его забыть, произойдет "проваливание" (fall-through), и код начнет выполняться для следующего `case` подряд, пока не встретит `break` или конец `switch`. Иногда это делают намеренно (как в примере с `case 3` и `case 4`), но чаще всего это ошибка.
- **`default`**: Необязательный блок. Срабатывает, если ни одно из значений `case` не совпало.

Пример кода

Давайте напишем программу, которая по номеру дня недели выводит его название.

C++

```
#include <iostream>

int main() {
    int day_of_week;
    std::cout << "Введите номер дня недели (1-7): ";
    std::cin >> day_of_week;

    switch (day_of_week) {
        case 1:
            std::cout << "Понедельник" << std::endl;
            break;
        case 2:
            std::cout << "Вторник" << std::endl;
            break;
        case 3:
            std::cout << "Среда" << std::endl;
            break;
        case 4:
            std::cout << "Четверг" << std::endl;
            break;
        case 5:
            std::cout << "Пятница" << std::endl;
            break;
        case 6:
        case 7:
            // Пример "проваливания" для выходных
            std::cout << "Выходной день!" << std::endl;
            break;
        default:
            std::cout << "Ошибка: неверный номер дня." << std::endl;
            break;
    }

    return 0;
}
```

---

13. Циклические конструкции в языке C++: синтаксис, блок-схема. Примеры. Основные правила использования циклов.

1) Что такое цикл — простыми словами

Цикл — конструкция, позволяющая повторять кусок кода несколько раз, пока выполняется некоторое условие. Это удобный инструмент для обхода массивов/контейнеров, повторения вычислений, организации ожидания ввода и т.п. В C++ есть несколько основных видов циклов: `for`, `while`, `do-while`, а также *range-based for* (специальный `for` для контейнеров). Плюс: алгоритмы STL (например, `std::for_each`, `std::accumulate`) часто лучше заменяют явные циклы.

---

2) Синтаксис (коротко)

`for` (обычный):

```
for (инициализация; условие; шаг) {
    // тело цикла
}
```

`while`:

```
while (условие) {
    // тело
}
```

`do-while`:

```
do {
    // тело (выполняется как минимум 1 раз)
} while (условие);
```

Range-based for (C++11+):

```
for (auto &elem : container) {
    // работает с каждым элементом container
}
```

3) Блок-схемы (ASCII — для понимания алгоритма)

`for` (общая блок-схема):

```
Start
  |
  v
инициализация (i = ... )
  |
  v
[условие i <= n ?] ---No---> End
      |
     Yes
      |
      v
   тело цикла
```

```
      |
      v
   шаг (i++)
      |
      v
   назад к проверке условия
```

while:

```
Start
  |
  v
[условие ?] ---No---> End
   |
  Yes
   |
   v
 тело
   |
   v
 обратно к условию
```

do-while:

```
Start
  |
  v
 тело (выполняется сразу)
  |
  v
 [условие ?] ---Yes---> тело
    |
   No
    |
   End
```

4) Примеры кода и разбор (пояснения встроены как комментарии)

Пример 1 — `for`: сумма чисел 1..N

```
#include <iostream>

int main() {
    int N = 10;
    int sum = 0;

    // for (инициализация; условие; шаг)
    for (int i = 1; i <= N; ++i) {
        sum += i; // каждый шаг добавляем i к сумме
    }

    std::cout << "Sum 1.." << N << " = " << sum << '\n';
    return 0;
}
```

Пояснение: `i` локальна внутри цикла; сначала выполняется инициализация `i = 1`, затем проверка `i <= N`. Если true — выполняется тело, затем ++i, снова проверка и т.д.

Пример 2 — `while`: чтение чисел до нуля

```
#include <iostream>

int main() {
    int x;
    std::cout << "Enter numbers (0 to stop):\n";
    // пока введено число и оно не 0 — печатаем
    while (std::cin >> x && x != 0) {
        std::cout << "You entered: " << x << '\n';
    }
    std::cout << "Done\n";
    return 0;
}
```

Пояснение: Условие проверяется *перед* первым выполнением тела; если ввод сразу плохой или 0 — тело не выполнится.

Пример 3 — `do-while`: меню или запрос, который должен выполниться хотя бы раз

```
#include <iostream>

int main() {
    char ans;
    do {
        std::cout << "Do you want to continue? (y/n): ";
        std::cin >> ans;
    } while (ans == 'y' || ans == 'Y'); // тело выполняется минимум один раз
    return 0;
}
```

Пример 4 — range-based for и изменение контейнера

```
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v {1,2,3,4,5};

    // Удобный цикл по элементам (изменяем элементы по ссылке)
    for (auto &e : v) {
        e *= 2; // умножаем каждый элемент на 2
    }

    for (auto e : v) std::cout << e << ' ';
    std::cout << '\n';
    return 0;
}
```

Пример 5 — вложенные циклы (таблица умножения) и break/continue

```
#include <iostream>

int main() {
    for (int i = 1; i <= 5; ++i) {
        for (int j = 1; j <= 5; ++j) {
            if (i * j > 12) break;     // выйти из внутреннего цикла
            if ((i * j) % 2 == 0) continue; // пропустить чётные результаты
            std::cout << i * j << '\t';
        }
        std::cout << '\n';
    }
    return 0;
```

```
}
```

5) Основные правила и рекомендации (важно для студентов)

1. **Инициализация, проверка, шаг**: мысленно разделяйте цикл на эти три части — это помогает избегать ошибок.
2. **Избегайте бесконечных циклов**, если не хотите специально ждать внешнего события: `for(;;)` или `while(true)` без `break` → бесконечность.
3. **Следите за областью видимости**: переменные, объявленные в `for (int i=0;...)`, видны только внутри цикла.
4. **Off-by-one ошибки**: проверьте, включительно ли условие (<=) или эксклюзивно (<).
5. **Подходящий тип индекса**: для индексов контейнеров используйте `std::size_t` (или `auto i = 0u`) при индексировании по `size()` — но будьте осторожны со смешиванием signed/unsigned.
6. **Не модифицируйте контейнер при итерации простым индексом/итератором**, если это меняет структуру (erase может инвалидировать итераторы). Для удаления используйте `erase` + `remove_if` или цикл с итератором, корректно обновляя итератор.
7. **Предпочитайте алгоритмы STL** (например, `std::accumulate`, `std::find`, `std::for_each`) — они понятнее и часто быстрее/безопаснее.
8. **Кеширование размера контейнера**: `for (size_t i=0, n=v.size(); i<n; ++i)` — раньше рекомендовали для списков, но современные компиляторы оптимизируют вызовы `size()`. Если `size()` — O(1) (как у вектора) — это не важно.
9. **Проверяйте сложность вложенных циклов**: вложенный цикл N×M может стоить дорого. Оцените алгоритм.
10. **Используйте `++i` вместо `i++` у итераторов по умолчанию (классические рекомендации)** — для встроенных типов разницы нет, но для пользовательских итераторов префиксная форма может быть эффективнее.

6) Частые ошибки

- Сравнение signed/unsigned (например `int i = -1; for (size_t j=0; j < v.size(); ++j)` — будьте аккуратны).
- Удаление элемента из `vector` по индексу внутри простого `for (size_t i=0; i<v.size(); ++i)` без корректировки индекса или использования `erase` + возвращаемого итератора.
- Опечатки в условии (`i <= n` vs `i < n`).
- Опирание на порядок вычисления в условии (в C++ есть последовательность выражений; не полагайтесь на побочные эффекты в условиях).

---

14. Функция: понятие, структурные единицы в синтаксисе. Типичный механизм передачи и возврата значений.

Понятие

Функция — это именованный, самодостаточный блок кода, предназначенный для выполнения определённой задачи. Представьте, что у вас есть сложная задача, например,

"приготовить ужин". Вы можете разбить её на подзадачи: "почистить картошку", "сварить суп", "пожарить мясо". Каждая из этих подзадач — это функция.

Основные преимущества функций:

- **Повторное использование кода**: Написав функцию один раз, вы можете вызывать её из разных мест программы сколько угодно раз.
- **Модульность и читаемость**: Программа, разбитая на небольшие логические функции, гораздо проще для понимания и отладки, чем одно сплошное полотно кода.
- **Абстракция**: Вы можете использовать функцию, не вникая в детали её реализации. Вам достаточно знать, *что* она делает, а не *как*.

Структурные единицы синтаксиса

Любая функция в C++ состоит из следующих частей:

```
тип_возвращаемого_значения имя_функции(список_параметров) { // тело_функции }
```

1. **Тип возвращаемого значения**: Тип данных, который функция "вернёт" после своего выполнения. Если функция ничего не возвращает, используется ключевое слово `void`.
2. **Имя функции**: Уникальный идентификатор, по которому к функции можно обратиться.
3. **Список параметров** (или аргументов): Переменные, которые функция принимает в качестве входных данных. Они указываются в круглых скобках через запятую, с указанием их типов. Если функция не принимает параметров, скобки остаются пустыми.
4. **Тело функции**: Блок кода в фигурных скобках `{}`. Здесь находится логика, которую выполняет функция.

Типичный механизм передачи и возврата значений

1. **Передача (Вызов функции)**: Когда вы вызываете функцию, вы передаете ей конкретные значения, называемые **аргументами**. Эти значения копируются в **параметры** функции (переменные в её скобках). `int result = sum(5, 10); // 5 и 10 - это аргументы`
2. **Возврат значения**: Внутри функции оператор `return` завершает её выполнение и отправляет указанное после него значение обратно в то место, откуда функция была вызвана. `return a + b; // Возвращаем результат сложения`

Возвращённое значение можно присвоить переменной, использовать в выражении или просто проигнорировать.

Пример кода

C++

```
#include <iostream>

// Определение функции sum.
// - Тип возвращаемого значения: int
// - Имя функции: sum
// - Параметры: int a, int b
```

```
int sum(int a, int b) {
    // Тело функции
    int result = a + b;
    return result; // Возвращаем вычисленное значение
}

// Функция, которая ничего не возвращает (void)
void printMessage(std::string message) {
    std::cout << message << std::endl;
    // return не нужен, либо можно написать return; для досрочного выхода
}

int main() {
    int x = 15;
    int y = 7;

    // 1. Вызов функции sum и сохранение результата
    int total = sum(x, y); // x и y - аргументы
    std::cout << "Сумма " << x << " и " << y << " равна " << total <<
std::endl;

    // 2. Вызов функции sum внутри другого выражения
    std::cout << "Еще одна сумма: " << sum(100, 200) << std::endl;

    // 3. Вызов void-функции
    printMessage("Привет из функции!");

    return 0;
}
```

---

15. Виды объявлений функций в C++. Многофайловые проекты. Примеры.

1) Главное различие: объявление (declaration) vs определение (definition)

- **Объявление (prototype)** сообщает компилятору: «у меня где-то есть такая функция — вот её имя, тип возвращаемого значения и параметры». Пример:

```
int add(int a, int b); // объявление (прототип)
```

- **Определение** даёт тело функции — реализацию:

```
int add(int a, int b) { return a + b; } // определение
```

Включение только объявления в заголовок позволяет подключающим файлам знать о функции, а реализация может находиться в отдельном `.cpp`.

2) Виды объявлений / форм функции (список с пояснениями)

1. **Прототип (forward declaration)**
   `int foo(double);` — используется в заголовках и перед использованием.
2. **Определение (function definition)**
   `int foo(double x) { /* ... */ }` — тело функции.
3. **Inline-функции**
   `inline int sq(int x) { return x*x; }` — можно определять в заголовке; допускается несколько одинаковых определений в разных единицах трансляции (ODR-исключение для `inline`).

4. **`constexpr` функции**
   `constexpr int cube(int x) { return x*x*x; }` — может быть вычислена во время компиляции, если аргументы — константы.
5. **Шаблонные функции (templates)**
   `template<typename T> T max(T a, T b) { return (a < b) ? b : a; }` — обязаны быть доступны при компиляции (обычно определяются в заголовках).
6. **Перегрузка функций (overloading)**
   Несколько объявлений с одним именем, но разными параметрами:
7. `void print(int);`
8. `void print(const std::string&);`
9. **Функции с аргументами по умолчанию**
   `int f(int a, int b = 10);` — значение по умолчанию задаётся в объявлении (обычно в заголовке).
10. **Внутренняя видимость (`static`/unnamed namespace)**
    `static int helper();` или `namespace { int helper(); }` — функция видна только в текущем .cpp (internal linkage).
11. **`extern "C"` (C linkage)**
    Для взаимодействия с C: `extern "C" void c_func(int);` — отключает манглинг имени.
12. **Члены класса (member functions)**
    Объявляются в теле класса, могут быть определены внутри (inline) или вне класса:
13. `struct S { void f(); };`
14. `void S::f() { /* определение */ }`
15. **Указатели на функции и `std::function`**
    Объявление типа указателя: `using FuncPtr = int(*)(int,int);`
16. **Lambda (функции на месте)**
    `auto lam = [](int x){ return x+1; };` — не классическое объявление, но часто используется вместо функций.

3) Практика: организация многофайлового проекта (правильный шаблон)

Правило: в заголовках (`.h`/`.hpp`) — декларации функций, inline/шаблоны/constexpr (то есть всё, что должно быть видно при компиляции). В `.cpp` — определения (реализация). Это уменьшает время компиляции и исключает множественные определения.

Пример: три файла (мини-библиотека + main)

math_utils.h

```
#pragma once
#include <vector>

// --- объявление (прототипы) ---
int add(int a, int b);
int factorial(int n);

// inline-функцию можно определить в заголовке
inline int square(int x) { return x * x; }

// constexpr тоже определяем в заголовке
constexpr int cube(int x) { return x * x * x; }

// шаблонная функция — определение в заголовке обязательно
template<typename T>
T max_template(T a, T b) {
    return (a < b) ? b : a;
```

```
}

// функция с параметром по умолчанию
double average(const std::vector<int>& data, int n = -1);
```

math_utils.cpp

```
#include "math_utils.h"
#include <numeric>

// определение обычных функций
int add(int a, int b) {
    return a + b;
}

int factorial(int n) {
    if (n < 0) return -1; // простая обработка ошибки
    int res = 1;
    for (int i = 2; i <= n; ++i) res *= i;
    return res;
}

double average(const std::vector<int>& data, int n) {
    if (data.empty()) return 0.0;
    int count = (n <= 0 || n > static_cast<int>(data.size())) ? data.size() :
n;
    double sum = std::accumulate(data.begin(), data.begin() + count, 0.0);
    return sum / count;
}
```

main.cpp

```
#include <iostream>
#include "math_utils.h"

int main() {
    std::cout << "add(2,3) = " << add(2,3) << '\n';
    std::cout << "square(4) = " << square(4) << '\n';
    std::cout << "cube(3) = " << cube(3) << '\n';
    std::cout << "max_template(3,7) = " << max_template(3,7) << '\n';
    std::vector<int> v{1,2,3,4,5};
    std::cout << "average = " << average(v) << '\n';
    return 0;
}
```

Компиляция (g++):

единый вызов:

```
g++ -std=c++17 main.cpp math_utils.cpp -o program
```

отдельная компиляция + линковка:

```
g++ -std=c++17 -c math_utils.cpp -o math_utils.o
g++ -std=c++17 -c main.cpp -o main.o
g++ math_utils.o main.o -o program
```

4) Частые ошибки при многофайловой организации и как их исправить

1. **"Undefined reference" (ошибка линковщика)** — вы объявили функцию в заголовке, но не определили её в .cpp или не слинковали соответствный .o. Решение: добавить определение или правильно собрать и связать объектные файлы.
2. **"Multiple definition"** — вы поместили определение обычной (не-inline) функции в заголовок и включили этот заголовок в несколько .cpp — при линковке будет несколько одинаковых определений. Решения:
   - Перенести определение в .cpp (оставить в заголовке только декларацию).
   - Либо пометить функцию `inline` (допускается одинаковое определение в разных единицах трансляции).
   - Или использовать `static` в заголовке (но `static` создаст отдельную копию в каждом .o и скрывает функцию локально — обычно не то, что нужно).
3. **Шаблоны и inline** — шаблонные функции должны быть определены в заголовке, потому что компилятор генерирует их по месту использования.
4. **Не ставить `using namespace std;` в заголовках** — это «заражает» все файлы, включающие заголовок.
5. **Default-параметры**: указывайте значения по умолчанию в объявлении (обычно в заголовке), но не в обеих — это может привести к неоднозначности. Значение по умолчанию используется компилятором в месте вызова.

5) Специальные случаи и полезные техники

- **Взаимная рекурсия (forward declaration):**

```
int even(int n);
int odd(int n);

int even(int n) {
    if (n == 0) return 1;
    return odd(n - 1);
}

int odd(int n) {
    if (n == 0) return 0;
    return even(n - 1);
}
```

  Здесь предварительное объявление необходимо, чтобы `even` видел `odd` и наоборот.

- **`extern "C"` для C API:**

```
extern "C" void c_function(int);
```

  Полезно при связывании с библиотеками на C (отключает «манглинг» имён).

- **Member functions**: объявлять в классе, определять вне класса с `ClassName::method` — это стандартная практика для читаемости.
- **std::function и указатели на функции** — позволяют передавать поведение как параметр:
- `using FuncPtr = int(*)(int,int);`
- `int operate(int a, int b, FuncPtr f) { return f(a,b); }`

6) Резюме и лучшие практики (коротко)

- В заголовках — **объявления**, inline/constexpr/шаблоны можно определять прямо в .h.
- В `.cpp` — реальные реализации функций.
- Используйте include-guards (`#pragma once` или классические `#ifndef/#define/#endif`).
- Избегайте определения (non-inline) функций в заголовках, чтобы не получить multiple definition.
- Для шаблонов — держите реализацию в заголовке (или в отдельном файле, который включается).
- Минимизируйте зависимости в заголовках (forward declarations, PIMPL, минимальные `#include`) → уменьшит время сборки.
- Понимайте разницу между declaration/definition — это ключ к пониманию ошибок компиляции/линковки.

---

16. Основные механизмы передачи параметров функции. Преимущества, недостатки. Примеры.

Способ, которым аргументы из вызывающего кода передаются в параметры функции, — это ключевой аспект. В C++ есть три основных механизма.

1. Передача по значению (Pass-by-Value)

Это механизм по умолчанию. Когда вы передаете переменную в функцию, **создается её полная копия**. Все изменения, которые происходят с параметром внутри функции, никак не влияют на исходную переменную.

Преимущества:

- **Безопасность**. Функция не может случайно испортить данные в вызывающем коде. Это защищает от побочных эффектов.

Недостатки:

- **Неэффективность для больших объектов**. Копирование больших структур, классов или контейнеров (как `std::vector`) может быть очень затратным по времени и памяти.

2. Передача по ссылке (Pass-by-Reference)

Чтобы использовать этот механизм, нужно добавить амперсанд (`&`) после типа параметра. В этом случае в функцию передается не копия, а **псевдоним (alias)** исходной переменной. Параметр становится вторым именем для той же ячейки памяти.

Преимущества:

- **Эффективность**. Никакого копирования не происходит, что идеально для больших объектов.
- **Возможность изменять исходные данные**. Это полезно, когда функция должна вернуть несколько значений или изменить состояние объекта.

Недостатки:

- **Потенциальная опасность**. Функция может изменить исходную переменную, даже если это не входило в ваши планы. Чтобы этого избежать, используют **константные ссылки** (`const T&`), которые запрещают модификацию.

3. Передача по указателю (Pass-by-Pointer)

В функцию передается **адрес переменной в памяти**. Чтобы получить доступ к значению по этому адресу, указатель нужно **разыменовать** с помощью звездочки (`*`).

Преимущества:

- **Эффективность**. Как и ссылка, не создает копий.
- **Возможность изменять исходные данные**.
- **Возможность передать `nullptr`**. Указатель может быть "нулевым", что означает, что он ни на что не указывает. Это позволяет передавать необязательные параметры. Ссылки не могут быть нулевыми.

Недостатки:

- **Более сложный синтаксис** (`*` для разыменования, `->` для доступа к членам).
- **Требует осторожности**. Работа с "сырыми" указателями может привести к ошибкам (например, разыменование `nullptr`).

Пример кода

C++

```
#include <iostream>

// 1. Передача по значению
void pass_by_value(int num) {
    num = 100; // Изменяем КОПИЮ
    std::cout << "Внутри pass_by_value, num = " << num << std::endl;
}

// 2. Передача по ссылке
void pass_by_reference(int& num_ref) {
    num_ref = 200; // Изменяем ОРИГИНАЛ
    std::cout << "Внутри pass_by_reference, num_ref = " << num_ref <<
std::endl;
}

// 3. Передача по указателю
void pass_by_pointer(int* num_ptr) {
    if (num_ptr != nullptr) { // Всегда проверяем указатель!
        *num_ptr = 300; // Изменяем ОРИГИНАЛ через разыменование
        std::cout << "Внутри pass_by_pointer, *num_ptr = " << *num_ptr <<
std::endl;
    }
}

int main() {
    int original_number = 10;
    std::cout << "Начальное значение: " << original_number << std::endl <<
std::endl;
```

```
    // --- Тест 1: передача по значению ---
    pass_by_value(original_number);
    std::cout << "После pass_by_value, оригинал = " << original_number <<
std::endl << std::endl;

    // --- Тест 2: передача по ссылке ---
    pass_by_reference(original_number);
    std::cout << "После pass_by_reference, оригинал = " << original_number <<
std::endl << std::endl;

    // Сбросим значение
    original_number = 10;
    std::cout << "Сбросили значение до: " << original_number << std::endl <<
std::endl;

    // --- Тест 3: передача по указателю ---
    pass_by_pointer(&original_number); // Передаем АДРЕС переменной
    std::cout << "После pass_by_pointer, оригинал = " << original_number <<
std::endl << std::endl;

    return 0;
}
```

---

17. Рекурсивные функции. Примеры и правила использования.

Понятие

Рекурсия — это процесс, при котором функция вызывает саму себя, либо напрямую, либо косвенно через другую функцию. Это мощный метод решения задач, которые можно разбить на более простые, но однотипные подзадачи.

Классическая аналогия — матрешка. Чтобы открыть матрешку, нужно открыть матрешку поменьше внутри нее. И так до тех пор, пока мы не дойдем до самой маленькой, которая уже не открывается.

Правила использования и структура рекурсивной функции

Чтобы рекурсия не стала бесконечной (и не привела к ошибке переполнения стека вызовов), любая рекурсивная функция должна иметь две обязательные части:

1. **Базовый случай (Base Case)**: Это условие, при котором рекурсия прекращается. Это "самая маленькая матрешка", которая решается без дальнейших рекурсивных вызовов. **Без базового случая рекурсия будет бесконечной.**
2. **Рекурсивный шаг (Recursive Step)**: Часть функции, где она вызывает саму себя, но с измененными параметрами, которые приближают ее к базовому случаю. Например, если функция обрабатывает число `n`, рекурсивный вызов обычно будет для `n-1`.

Как это работает? (Стек вызовов)

При каждом вызове функции (включая рекурсивный) в специальной области памяти, называемой **стеком**, создается новый "кадр" (stack frame), где хранятся её локальные переменные и параметры. Когда функция завершается, её кадр из стека удаляется. В рекурсии кадры накладываются друг на друга, как стопка тарелок. Когда достигается

базовый случай, начинается обратный процесс: функции завершаются одна за другой, возвращая свои результаты на предыдущий уровень, и "стопка" разбирается.

Пример: вычисление факториала

Факториал числа `n` (обозначается как `n!`) — это произведение всех натуральных чисел от 1 до `n`. `5! = 5 * 4 * 3 * 2 * 1 = 120`

Рекурсивное определение факториала:

- `n! = n * (n-1)!` (рекурсивный шаг)
- `0! = 1` (базовый случай)

Давайте реализуем это в коде.

C++

```
#include <iostream>

long long factorial(int n) {
    // Проверка на корректность ввода
    if (n < 0) {
        return -1; // Факториал отрицательных чисел не определен
    }

    // 1. Базовый случай
    if (n == 0) {
        std::cout << "Базовый случай: n = 0, возвращаем 1" << std::endl;
        return 1;
    }
    // 2. Рекурсивный шаг
    else {
        std::cout << "Рекурсивный шаг: n = " << n << ". Вызываем factorial("
<< n - 1 << ")" << std::endl;
        return n * factorial(n - 1);
    }
}

int main() {
    int number = 4;
    std::cout << "Вычисление факториала для " << number << ":" << std::endl;
    long long result = factorial(number);

    if (result != -1) {
        std::cout << "\nРезультат: " << number << "! = " << result <<
std::endl;
    } else {
        std::cout << "Неверный ввод." << std::endl;
    }

    return 0;
}
```

Трассировка вызова `factorial(4)`:

1. `factorial(4)` вызывает `factorial(3)` и ждет результат.
2. `factorial(3)` вызывает `factorial(2)` и ждет.
3. `factorial(2)` вызывает `factorial(1)` и ждет.
4. `factorial(1)` вызывает `factorial(0)` и ждет.

5. `factorial(0)` — это базовый случай. Он немедленно возвращает `1`.
6. `factorial(1)` получает `1`, умножает на свой `n` (1) и возвращает `1 * 1 = 1`.
7. `factorial(2)` получает `1`, умножает на свой `n` (2) и возвращает `2 * 1 = 2`.
8. `factorial(3)` получает `2`, умножает на свой `n` (3) и возвращает `3 * 2 = 6`.
9. `factorial(4)` получает `6`, умножает на свой `n` (4) и возвращает `4 * 6 = 24`.

---

18. Описание статического массива. Одномерные массивы. Адресация. Варианты перебора элементов. Примеры.

Описание статического массива

Массив — это структура данных, которая хранит фиксированное количество элементов **одного и того же типа**. "Статический" означает, что его размер должен быть известен на этапе компиляции и не может быть изменен во время выполнения программы.

Ключевые свойства:

- **Фиксированный размер**: `int arr[10];` — массив ровно на 10 целых чисел.
- **Однородность**: Все элементы имеют один тип. Нельзя в `int` массив положить `double` или `string`.
- **Непрерывное хранение в памяти**: Элементы массива располагаются в памяти вплотную, один за другим. Это обеспечивает очень быстрый доступ к любому элементу по его индексу.

Одномерные массивы и адресация

Одномерный массив можно представить как ряд ячеек. Доступ к каждой ячейке осуществляется по **индексу** — целому числу, которое указывает на позицию элемента. **Нумерация индексов в C++ всегда начинается с 0.**

Для массива `int arr[5];`:

- Первый элемент: `arr[0]`
- Второй элемент: `arr[1]`
- ...
- Последний элемент: `arr[4]`

Адресация: Поскольку элементы лежат в памяти подряд, адрес любого элемента `arr[i]` можно вычислить по формуле: `Адрес(arr[i]) = Адрес(arr[0]) + i * sizeof(тип_элемента)` Это вычисление компилятор делает автоматически и очень быстро.

Варианты перебора элементов

Существует несколько стандартных способов пройтись по всем элементам массива.

1. **Классический цикл `for` с индексом**: Самый универсальный способ. Дает полный контроль над процессом.

C++

```
for (int i = 0; i < РАЗМЕР_МАССИВА; ++i) {
    // работаем с arr[i]
}
```

2. **Range-based for loop (цикл по диапазону)**: Более современный, краткий и безопасный синтаксис (появился в C++11). Идеален, когда нужно просто перебрать все элементы от начала до конца.

   C++

   ```
   for (int element : arr) {
       // работаем с element (копией элемента массива)
   }
   // Для изменения элементов массива используйте ссылку:
   for (int& element : arr) {
       element = 0; // изменяем сам элемент в массиве
   }
   ```

3. **Перебор с определенным шагом**: Классический `for` легко модифицировать для перебора, например, каждого второго элемента.

   C++

   ```
   // Перебираем элементы с индексами 0, 2, 4, ...
   for (int i = 0; i < РАЗМЕР_МАССИВА; i += 2) {
       // работаем с arr[i]
   }
   ```

Пример кода

C++

```
#include <iostream>

int main() {
    // Объявление и инициализация статического массива из 5 целых чисел
    const int SIZE = 5;
    int numbers[SIZE] = {10, 20, 30, 40, 50};

    // --- 1. Перебор с помощью классического for ---
    std::cout << "1. Классический for:" << std::endl;
    for (int i = 0; i < SIZE; ++i) {
        std::cout << "Элемент с индексом " << i << " = " << numbers[i] <<
std::endl;
        // Изменим значение для следующего примера
        numbers[i] += 1;
    }

    // --- 2. Перебор с помощью range-based for ---
    std::cout << "\n2. Range-based for (чтение):" << std::endl;
    for (int val : numbers) {
        std::cout << val << " ";
    }
    std::cout << std::endl;

    // --- 3. Изменение элементов с помощью range-based for и ссылки ---
    std::cout << "\n3. Умножаем каждый элемент на 2:" << std::endl;
    for (int& val : numbers) {
```

```
        val *= 2;
    }
    // Распечатаем результат
    for (int val : numbers) {
        std::cout << val << " ";
    }
    std::cout << std::endl;

    // --- 4. Перебор каждого второго элемента ---
    std::cout << "\n4. Каждый второй элемент:" << std::endl;
    for (int i = 0; i < SIZE; i += 2) {
        std::cout << "Элемент с индексом " << i << " = " << numbers[i] <<
std::endl;
    }

    return 0;
}
```

---

19. Многомерные статические массивы. Адресация, инициализация. Примеры.

Понятие

Многомерный массив — это, по сути, "массив массивов". Самый распространенный вид — двумерный массив, который удобно представлять в виде таблицы или матрицы, состоящей из строк и столбцов.

Например, `int matrix[3][4];` — это массив из 3 элементов, где каждый элемент, в свою очередь, является массивом из 4 `int`. Мы получаем таблицу 3x4.

Инициализация

Инициализировать многомерный массив можно с помощью вложенных фигурных скобок. Внешние скобки относятся ко всему массиву, а каждая внутренняя пара скобок — к одной "строке".

C++

```
// Инициализация матрицы 3x4
int matrix[3][4] = {
    {1, 2, 3, 4},   // 0-я строка
    {5, 6, 7, 8},   // 1-я строка
    {9, 10, 11, 12} // 2-я строка
};
```

Если в инициализаторе указано меньше значений, оставшиеся элементы будут инициализированы нулями.

Адресация и хранение в памяти

Несмотря на то, что мы представляем двумерный массив как таблицу, в памяти компьютера он хранится **линейно (в одну строчку)**, по так называемому "строчно-ориентированному" (row-major) порядку. Сначала идут все элементы нулевой строки, затем — все элементы первой строки и так далее.

Для `matrix[3][4]`: `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]`

Доступ к элементу `matrix[i][j]` (элемент в `i`-й строке и `j`-м столбце) компилятор вычисляет по формуле, зная количество столбцов: `Адрес(matrix[i][j]) = Базовый_адрес + (i * КОЛ-ВО_СТОЛБЦОВ + j) * sizeof(тип_элемента)` Именно поэтому при передаче многомерного массива в функцию (см. вопрос 20) нужно указывать размеры всех измерений, кроме первого.

Пример кода

В примере мы создадим двумерный массив, заполним его значениями (таблица умножения) и выведем на экран.

C++

```
#include <iostream>
#include <iomanip> // для форматированного вывода setw

int main() {
    // Объявляем константы для размеров
    const int ROWS = 5;
    const int COLS = 5;

    // Объявляем и инициализируем нулями двумерный массив
    int multiplication_table[ROWS][COLS] = {0};

    // --- Заполнение массива ---
    // Используем вложенные циклы. Внешний идет по строкам.
    for (int i = 0; i < ROWS; ++i) {
        // Внутренний идет по столбцам в текущей строке.
        for (int j = 0; j < COLS; ++j) {
            // Элементу в i-й строке и j-м столбце присваиваем значение
            multiplication_table[i][j] = (i + 1) * (j + 1);
        }
    }

    // --- Вывод массива на экран в виде таблицы ---
    std::cout << "Таблица умножения:" << std::endl;
    for (int i = 0; i < ROWS; ++i) {
        for (int j = 0; j < COLS; ++j) {
            // setw(4) задает ширину поля в 4 символа для красивого вывода
            std::cout << std::setw(4) << multiplication_table[i][j];
        }
        std::cout << std::endl; // Переход на новую строку после каждой
строки матрицы
    }

    return 0;
}
```

---

20. Передача параметров одномерного массива (функции). Передача параметров двумерного массива (функции).

Понятие

Когда вы передаете статический массив в функцию, на самом деле происходит не копирование всего массива. Вместо этого в функцию передается **указатель на первый элемент массива**. Этот процесс называется "распадом массива до указателя" (array decay).

Следствия:

1. **Эффективность**: Не происходит копирования сотен или тысяч элементов.
2. **Потеря информации о размере**: Внутри функции невозможно узнать исходный размер массива. Поэтому размер **всегда нужно передавать вторым, отдельным параметром**.
3. **Изменение оригинала**: Так как функция получает указатель на исходные данные, любые изменения элементов массива внутри функции **отразятся на оригинальном массиве**.

Передача одномерного массива

Существует два синтаксически эквивалентных способа объявить параметр-массив в функции:

1. `void func(int arr[], int size);` — синтаксис с квадратными скобками, более наглядный.
2. `void func(int* arr, int size);` — синтаксис с указателем, который точнее отражает происходящее.

Компилятор воспринимает оба варианта абсолютно одинаково.

Пример кода:

C++

```
#include <iostream>

// Функция для печати элементов одномерного массива.
// arr[] - это на самом деле int* arr
void print_array(const int arr[], int size) { // const, т.к. мы только читаем
    std::cout << "Массив внутри функции: ";
    for (int i = 0; i < size; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

// Функция, которая изменяет массив
void modify_array(int arr[], int size) {
    for (int i = 0; i < size; ++i) {
        arr[i] *= 10;
    }
}

int main() {
    const int SIZE = 5;
    int my_array[SIZE] = {1, 2, 3, 4, 5};

    std::cout << "Массив до вызова функций:" << std::endl;
    print_array(my_array, SIZE);

    modify_array(my_array, SIZE);
```

```
    std::cout << "Массив после вызова modify_array:" << std::endl;
    print_array(my_array, SIZE);

    return 0;
}
```

Передача двумерного массива

Здесь ситуация сложнее. Как мы помним из вопроса 19, для вычисления адреса элемента `matrix[i][j]` компилятору нужно знать **количество столбцов**. Поэтому при передаче многомерного массива в функцию вы **обязаны** указать размеры всех измерений, кроме самого первого.

Синтаксис: `void func(int arr[][КОЛ-ВО_СТОЛБЦОВ], int кол-во_строк);`

Например, для массива `int matrix[3][4]` параметр функции должен выглядеть как `int arr[][4]`.

Пример кода:

C++

```
#include <iostream>

// Константа для количества столбцов
const int COLS = 4;

// Функция для печати двумерного массива
// Мы должны указать размер второго измерения (COLS)
void print_2d_array(const int arr[][COLS], int rows) {
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < COLS; ++j) {
            std::cout << arr[i][j] << "\t"; // \t - символ табуляции
        }
        std::cout << std::endl;
    }
}

int main() {
    const int ROWS = 3;
    int my_matrix[ROWS][COLS] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    std::cout << "Печать 2D-массива из функции:" << std::endl;
    print_2d_array(my_matrix, ROWS);

    return 0;
}
```

---

21. Организация и хранение данных в памяти компьютера. Адресация. Стек.

Пояснение (лекция — простым языком).

1. **Области памяти процесса**
    - **Текст (code)** — машинные инструкции программы (обычно доступно только для чтения).
    - **Данные (data)** — инициализированные глобальные и статические переменные.
    - **BSS** — неинициализированные глобальные/статические переменные (заполняются нулями при старте).
    - **Куча (heap)** — область для динамической памяти, управляемой программистом (new/malloc). Память выделяется/освобождается в произвольном порядке.
    - **Стек (stack)** — автоматически управляемая область для локальных переменных, параметров функций и адресов возврата. Работает по принципу LIFO (last in — first out).
2. **Адресация**
    - **Адрес** — уникальный номер ячейки памяти (обычно байта). В языке C++ указатель — переменная, хранящая адрес.
    - **Линейная/виртуальная адресация:** ОС и аппарат создают виртуальное адресное пространство для процесса; физическая адресация скрыта (страничная организация и MMU).
    - **Выравнивание (alignment):** типы данных обычно выровнены (например, `int` на границе 4 байт). Это влияет на адреса и производительность.
    - **Память и указатели:** арифметика указателей в C++ оперирует элементами типа (например, `ptr + 1` сдвигает на sizeof(type)).
3. **Стек и стековые кадры (stack frames)**
    - Для каждого вызова функции в стеке создаётся кадр: локальные переменные, параметры, адрес возврата, возможно сохраняемые регистры.
    - При возврате кадр снимается — локальные переменные уничтожаются автоматически.
    - **Переполнение стека (stack overflow)** происходит при глубокой рекурсии или слишком больших автоматических переменных.
4. **Практические замечания**
    - Большие буферы и массивы лучше размещать в куче, чтобы не раздуть стек.
    - Используйте RAII (std::vector, unique_ptr) для динамической памяти, чтобы избежать утечек.
    - Не используйте адреса локальных переменных после возврата функции — это UB.

Пример кода (демонстрация областей и стека):

```
#include <iostream>
#include <vector>

int global_init = 42;    // data (инициализированные данные)
static int global_uninit; // bss (неинициализированные данные)

void foo(int x) {
    int local = x * 2;    // локальная переменная в стеке
    int arr[10];          // массив на стеке (малый)
    std::cout << "Адрес локальной переменной local: " << &local << "\n";
    std::cout << "Адрес массива arr: " << arr << "\n";
}

int main() {
    std::cout << "Адрес кода (функция main): " << (void*) &main << "\n";
```

```
    std::cout << "Адрес глобальной переменной global_init: " << &global_init
<< "\n";
    std::cout << "Адрес глобальной переменной global_uninit: " <<
&global_uninit << "\n";

    foo(7);

    // Большой буфер лучше в куче:
    std::vector<int> big(1000000); // управляющие данные в стеке, буфер в
куче
    std::cout << "Размер вектора big: " << big.size() << "\n";

    return 0;
}
```

---

22. Понятие указатели и динамическая память. Инициализация указателей (4 способа). Примеры.

Пояснение.

1. **Понятие указателя**
   - Указатель — переменная, которая хранит адрес другой переменной (или области памяти). Тип `T*` означает «указатель на `T`».
   - Разыменование (`*p`) позволяет читать/писать по адресу.
2. **Динамическая память (куча)**
   - В C++ выделение: `new T` / `new T[n]` (или `std::malloc` в C-стиле). Освобождение: `delete` / `delete[]`.
   - Современный C++ рекомендует использовать умные указатели (`std::unique_ptr`, `std::shared_ptr`) и контейнеры (`std::vector`) вместо «сырого» `new`/`delete`.
3. **Четыре способа инициализации указателей**
   - **Неинициализированный указатель** (опасно): `int *p;` — содержит «мусор» (не использовать до присвоения).
   - **Инициализация нулевым указателем**: `int *p = nullptr;` — безопасно, означает «не указывает никуда».
   - **Указатель на существующую переменную**: `int x = 5; int *p = &x;` — указывает на `x`.
   - **Выделение динамической памяти**: `int *p = new int(10);` или `int *arr = new int[n];` — указывает на блок в куче.
4. **Советы по безопасности**
   - Проверяйте `nullptr` перед разыменованием.
   - Освобождайте память при необходимости или используйте RAII.
   - После `delete p;` хорошая практика — присвоить `p = nullptr;` чтобы избежать «висячих указателей».

Пример кода:

```
#include <iostream>
#include <memory>

int main() {
    // 1) неинициализированный (опасно)
    int *p1; // p1 содержит неопределённое значение
```

```
    // 2) nullptr (безопасно)
    int *p2 = nullptr;
    if (p2 == nullptr) std::cout << "p2 is null\n";

    // 3) указатель на существующую переменную
    int x = 100;
    int *p3 = &x;
    std::cout << "p3 -> " << *p3 << " (адрес " << p3 << ")\n";

    // 4) динамическая память
    int *p4 = new int(42);
    std::cout << "p4 -> " << *p4 << " (адрес " << p4 << ")\n";
    delete p4;
    p4 = nullptr; // безопаснее

    // Современный вариант: умный указатель
    auto up = std::make_unique<int>(55);
    std::cout << "unique_ptr -> " << *up << "\n";

    return 0;
}
```

---

23. Операции с указателями. Преобразование указателей.

Пояснение.

1. **Основные операции с указателями**
   - **Присваивание адреса:** `p = &x;`
   - **Разыменование:** `*p` — доступ к объекту.
   - **Арифметика указателей:** для указателя `T*` выражение `p + k` сдвигает на `k * sizeof(T)` байт; `p++` перемещает к следующему элементу типа `T`. Полезно при работе с массивами.
   - **Сравнение указателей:** допустимо сравнивать указатели, указывающие на элементы одного массива (или один и тот же объект).
   - **Получение адреса элемента:** `&arr[i]` эквивалентно `arr + i`.
2. **Преобразование указателей**
   - **Неявное** преобразование допустимо от `T*` к `const T*` (без `const_cast`).
   - **Явные преобразования:** `reinterpret_cast`, `static_cast`, `const_cast`.
     - `static_cast<T*>` безопасен для некоторых иерархий типов (наследование).
     - `reinterpret_cast` превращает битовый паттерн адреса — низкоуровневое, платформозависимо; использовать осторожно.
     - `const_cast` убирает/добавляет `const` (опасно, если исходный объект действительно `const`).
   - **void*** — универсальный указатель в C/C++: `void*` может хранить адрес любого объекта, но для доступа нужно преобразовать обратно к `T*`.
3. **Правила и опасности**
   - Нельзя разыменовывать `nullptr` или неинициализированный указатель — UB.
   - Преобразование указателя к несовместимому типу и разыменование — UB.
   - Арифметика указателей корректна только внутри одного массива (включая «one past the end» позицию).

Пример кода (операции и преобразования):

```
#include <iostream>

struct A { int a; };
struct B { int b; };

int main() {
    int arr[5] = {1,2,3,4,5};
    int *p = arr;        // указывает на arr[0]
    std::cout << "p[2] = " << p[2] << "\n"; // арифметика указателей

    // Сдвиги
    p += 3;
    std::cout << "*p = " << *p << " (arr[3])\n";

    // void* как универсальный контейнер адреса
    void *pv = &arr[1];
    int *p_int = static_cast<int*>(pv); // приведение назад
    std::cout << "p_int points to " << *p_int << "\n";

    // reinterpret_cast — низкоуровневое приведение (обычно НЕ нужно)
    A a{10};
    B *pb = reinterpret_cast<B*>(&a); // небезопасно в общем случае
    std::cout << "reinterpret_cast pb->b (битовое представление) = " << pb->b
<< "\n";

    // const_cast
    const int ci = 7;
    const int *pci = &ci;
    // int *pi = const_cast<int*>(pci); // UB, если менять ci через pi
    // *pi = 8; // потенциально UB

    return 0;
}
```

---

24. Указатели и одномерные динамические массивы.

Пояснение.

1. **Выделение и освобождение**
   - `T* arr = new T[n];` — выделяет массив из `n` элементов типа `T`.
   - Освобождение: `delete[] arr;`. Нельзя использовать `delete arr;` — это UB.
2. **Доступ к элементам**
   - `arr[i]` и `*(arr + i)` эквивалентны. Указатель `arr` указывает на первый элемент (тип `T*`).
   - Можно пройти по массиву арифметикой указателя: `for (T* p = arr; p != arr + n; ++p) ....`
3. **Почему предпочитают std::vector**
   - `std::vector<T>` управляет памятью и размерами автоматически, обрабатывает исключения, предоставляет методы `size()`, `capacity()`, `reserve()`, `push_back()`.
   - В большинстве случаев используйте `std::vector`, а не «сырые» массивы.
4. **Особенности**
   - Память непрерывна (как в `malloc`).
   - При выделении большого массива возможны ошибки (bad_alloc) — обрабатывайте исключения при необходимости.

Пример кода (динамический массив):

```
#include <iostream>
#include <memory>

int main() {
    size_t n = 5;
    int *arr = new int[n]; // выделили массив из 5 int

    // инициализация
    for (size_t i = 0; i < n; ++i) arr[i] = static_cast<int>(i * 10);

    // вывод двумя способами
    for (size_t i = 0; i < n; ++i)
        std::cout << "arr[" << i << "] = " << arr[i] << "\n";

    // проход через указатель
    for (int *p = arr; p != arr + n; ++p)
        std::cout << *p << " ";
    std::cout << "\n";

    delete[] arr; // не забыть освободить

    // Современный стиль: уникальный указатель с массивом
    std::unique_ptr<int[]> up(new int[n]);
    for (size_t i = 0; i < n; ++i) up[i] = static_cast<int>(i + 1);
    // память очистится автоматически при выходе из области видимости

    return 0;
}
```

---

25. Указатели и многомерные динамические массивы.

Пояснение.

1. **Виды представления многомерного массива**
   - **Сплошной (contiguous) блок:** выделить `n*m` элементов и индексировать `(i*m + j)` — хранится как одна непрерывная область (эффективно, кэш-дружелюбно).
   - **Массив указателей (масив строк):** выделить массив указателей `T** rows = new T*[n];` затем для каждой строки `rows[i] = new T[m];`. Это даёт `n` отдельных блоков (не непрерывно), но удобную индексную нотацию `rows[i][j]`. Требует `n+1` вызовов `new` и соответствующих `delete[]`.
2. **Рекомендация**
   - Предпочитайте `std::vector` с одним блоком или `vector<vector<T>>` (второй удобен, но строки могут быть не смежны). Для истинно 2D с хорошей производительностью — `std::vector<T> data(n*m)` и вспомогательная функция доступа.
3. **Плюсы/минусы**
   - `T**` (каждая строка отдельно): простая запись `rows[i][j]`, но фрагментированная память и больше управления.
   - `T*` единый блок: эффективнее, но чуть менее интуитивно индексировать.

Пример кода (оба варианта):

```
#include <iostream>
```

```
#include <vector>

int main() {
    size_t n = 3, m = 4;

    // Вариант 1: единый непрерывный блок
    std::vector<int> data(n * m);
    // Инициализация
    for (size_t i = 0; i < n; ++i)
        for (size_t j = 0; j < m; ++j)
            data[i * m + j] = static_cast<int>(i * 10 + j);

    // Доступ
    std::cout << "Contiguous block:\n";
    for (size_t i = 0; i < n; ++i) {
        for (size_t j = 0; j < m; ++j)
            std::cout << data[i * m + j] << " ";
        std::cout << "\n";
    }

    // Вариант 2: массив указателей (каждая строка отдельный блок)
    int **rows = new int*[n];
    for (size_t i = 0; i < n; ++i)
        rows[i] = new int[m];

    // Инициализация
    for (size_t i = 0; i < n; ++i)
        for (size_t j = 0; j < m; ++j)
            rows[i][j] = static_cast<int>(i * 100 + j);

    std::cout << "Array of pointers:\n";
    for (size_t i = 0; i < n; ++i) {
        for (size_t j = 0; j < m; ++j)
            std::cout << rows[i][j] << " ";
        std::cout << "\n";
    }

    // Очистка памяти
    for (size_t i = 0; i < n; ++i) delete[] rows[i];
    delete[] rows;

    return 0;
}
```

---

26. Понятие, история и характеристика STL. Основы STL. Структура и назначение. Контейнеры.

Пояснение.

1. **Что такое STL**
   - STL (Standard Template Library) — часть стандартной библиотеки C++, включающая **контейнеры**, **итераторы**, **алгоритмы** и **функторы/предикаты**. Позволяет писать обобщённый код.
2. **Краткая история (без деталей дат)**
   - Концепция обобщённого программирования и реализация контейнеров/алгоритмов была предложена академически/индустриально, затем часть идей вошла в стандарт C++ (STL стал предтечей `<algorithm>`, `<vector>`, `<map>` и т.д.).
3. **Основные компоненты STL**

- **Контейнеры** — структуры для хранения элементов (`vector`, `list`, `set`, `map`, ...).
- **Итераторы** — абстрактный интерфейс для прохода по элементам контейнера (подобны указателям).
- **Алгоритмы** — шаблоны функций для сортировки, поиска, копирования (`std::sort`, `std::find`, `std::transform` и т.д.). Они работают с итераторами.
- **Адаптеры** — контейнерные/итераторные адаптеры (`stack`, `queue`, `priority_queue`, `reverse_iterator`).

4. **Назначение**
   - Повысить переиспользуемость кода, абстрагировать от конкретных типов и реализаций. Код алгоритмов отделён от конкретного контейнера.
5. **Характеристика**
   - STL — шаблонная, типобезопасная, эффективная по производительности (в большинстве случаев).
   - Контракты (итераторы, сложности операций) важны при выборе контейнера и алгоритма.

Пример кода (использование контейнеров и алгоритмов):

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> v = {5, 1, 4, 3, 2};

    // Алгоритм сортировки из STL
    std::sort(v.begin(), v.end());

    std::cout << "Sorted: ";
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";

    // Поиск
    auto it = std::find(v.begin(), v.end(), 3);
    if (it != v.end()) std::cout << "Found 3 at index " <<
std::distance(v.begin(), it) << "\n";

    return 0;
}
```

---

27. Передача итераторов в качестве параметров функции. Реализация программы по распечатке контейнера в диапазоне итераторов.

Пояснение.

1. **Идея**
   - Алгоритмы в STL оперируют итераторами. Функция, принимающая пару итераторов `[begin, end)`, становится обобщённой и может работать с любым контейнером, предоставляющим итераторы того же типа.
2. **Преимущества**
   - Универсальность: одна функция для `vector`, `list`, `deque`, `set` и т.д.
   - Безопасность: итераторы определяют границы диапазона (`[first, last)`).

3. **Практика**
    - Функция-шаблон: `template<typename It> void print_range(It first, It last)`.
    - Итератор проходится в цикле `for (; first != last; ++first)`.

Пример кода (универсальная печать):

```
#include <iostream>
#include <vector>
#include <list>
#include <iterator> // std::ostream_iterator (при желании)

template<typename Iterator>
void print_range(Iterator first, Iterator last, const char* sep = " ") {
    for (; first != last; ++first) {
        std::cout << *first;
        if (std::next(first) != last) std::cout << sep;
    }
    std::cout << "\n";
}

int main() {
    std::vector<int> v = {1,2,3,4};
    std::list<int>  l = {10,20,30};

    std::cout << "Vector: ";
    print_range(v.begin(), v.end());

    std::cout << "List: ";
    print_range(l.begin(), l.end(), " | ");

    // Можно печатать поддиапазоны:
    std::cout << "Subrange of vector (1..3): ";
    print_range(v.begin()+1, v.begin()+4); // работает для
RandomAccessIterator

    return 0;
}
```

---

28. Последовательные контейнеры: основные виды, характеристика.

Пояснение (основные последовательные контейнеры в STL):

1. **std::vector**
    - Динамический массив, хранит элементы в непрерывной памяти.
    - Быстрый доступ по индексу O(1), добавление в конец амортизированно O(1). Вставка/удаление в середине — O(n).
2. **std::deque**
    - Двусторонняя очередь, реализована как набор блоков. Поддерживает быстрые вставки/удаления в начале и в конце. Доступ по индексу — O(1) (но не обязательно в одной непрерывной области).
3. **std::list** (двусвязный список)
    - Каждый элемент содержит ссылки на соседей. Быстрая вставка/удаление в произвольной позиции, если есть итератор (O(1)), но доступ по индексу — O(n). Итераторы не инвалидируются при вставке/удалении других элементов (кроме удалённых).
4. **std::forward_list** (односвязный список)

    - Лёгкая версия списка, экономит память на указателях назад, полезен для однопроходных алгоритмов.
5. **std::array<T,N>**
    - Статический массив фиксированного размера (время компиляции), хранится непрерывно, имеет интерфейс контейнера.
6. **Когда какой выбирать**
    - `vector` — чаще всего: лучший выбор для большинства задач.
    - `deque` — когда нужно часто добавлять/удалять элементы с обоих концов.
    - `list` — когда часто вставляете/удаляете в середине и у вас много итераторов, которые не должны инвалидироваться.
    - `array` — когда размер известен на этапе компиляции и нужен стек/встроенный буфер.

Пример кода (демонстрация работы):

```
#include <iostream>
#include <vector>
#include <deque>
#include <list>
#include <array>

int main() {
    std::vector<int> v = {1,2,3};
    v.push_back(4); // append
    std::cout << "vector[2] = " << v[2] << "\n";

    std::deque<int> dq;
    dq.push_front(10);
    dq.push_back(20);
    std::cout << "deque front/back: " << dq.front() << "/" << dq.back() << 
"\n";

    std::list<int> lst = {1,2,3};
    auto it = lst.begin();
    ++it;
    lst.insert(it, 99); // вставка перед it
    std::cout << "list: ";
    for (int x : lst) std::cout << x << " ";
    std::cout << "\n";

    std::array<int,3> arr = {7,8,9};
    std::cout << "array size: " << arr.size() << "\n";

    return 0;
}
```

---

29. Ассоциативные контейнеры: основные виды, характеристика.

Пояснение.

Ассоциативные контейнеры хранят пары ключ–значение или только ключи и позволяют быстрый поиск по ключу.

1. **Упорядоченные (ordered)** — `std::set`, `std::map`, `std::multiset`, `std::multimap`
    - Обычно реализованы как сбалансированные деревья (например, красно-чёрное дерево).

- Ключи хранятся в отсортированном порядке.
- Операции поиска, вставки, удаления — O(log n).
- `std::map<Key, Value>` хранит пары (ключ → значение). `std::set<Key>` хранит только ключи. `multiset`/`multimap` допускают дублирующиеся ключи.

2. **Неупорядоченные (hash-based)** — `std::unordered_set`, `std::unordered_map`, `std::unordered_multimap`, `std::unordered_multiset`
   - Реализованы на основе хеш-таблиц.
   - Средняя сложность операций — O(1), в худшем случае — O(n) (при плохом хешировании).
   - Порядок элементов не определён (зависит от хешей и числа бакетов).
3. **Выбор между ними**
   - Нужен упорядоченный обход → `map`/`set`.
   - Нужен быстрый доступ по ключу и порядок не важен → `unordered_map`/`unordered_set`.
   - Нужны дубликаты ключей → `multimap`/`multiset`/`unordered_multimap`.
4. **Особенности**
   - Ключи в `map` обычно `const` (нельзя изменять ключ элемента в контейнере, потому что это нарушит структуру дерева).
   - hash-функции и equality для `unordered_*` можно задавать свои.

Пример кода (map и unordered_map):

```
#include <iostream>
#include <map>
#include <unordered_map>

int main() {
    std::map<std::string, int> age;
    age["Alice"] = 30;
    age["Bob"] = 25;
    for (const auto &p : age) // порядок по ключу (лексикографически)
        std::cout << p.first << " -> " << p.second << "\n";

    std::unordered_map<std::string, int> uage;
    uage["Alice"] = 30;
    uage["Bob"] = 25;
    std::cout << "unordered lookup Bob: " << uage.at("Bob") << "\n";

    return 0;
}
```

---

30. Контейнер `vector`. Назначение, инициализация, принцип работы.

Пояснение.

1. **Назначение**
   - `std::vector<T>` — динамический массив, предоставляющий случайный доступ, автоматическое управление размером и памятью.
2. **Инициализация (несколько способов)**
   - `std::vector<int> v;` — пустой вектор.
   - `std::vector<int> v(10);` — 10 элементов, проинициализированных значениями по умолчанию (0 для встроенных типов в C++11+? — для `int` не инициализирован в `vector<int>(10)`? На самом деле `vector<int>(10)`

инициализирует нулями; но лучше сказать: инициализировано значением `T()`).
    - `std::vector<int> v = {1,2,3};` — list-инициализация.
    - `std::vector<int> v(n, value);` — `n` элементов, каждый равен `value`.
    - `std::vector<int> v2(v1);` — копирование.
3. **Принцип работы: size vs capacity**
    - `size()` — текущее число элементов.
    - `capacity()` — сколько элементов может храниться без перевыделения памяти.
    - При `push_back` если `size() == capacity()`, `vector` перевыделяет память (обычно увеличивает capacity в 2 раза), копирует/перемещает элементы и освобождает старую память. Это даёт амортизированную O(1) сложность вставки в конец.
4. **Особенности**
    - Память `vector` — непрерывная → совместим с C API (указатель `&v[0]` или `v.data()`).
    - Резервирование мощности: `v.reserve(n)` уменьшает число перевыделений.
    - `v.shrink_to_fit()` предлагает уменьшить `capacity` до `size`.
    - При вставке/удалении не в конце — возможна инвалидизация итераторов и сдвиг элементов O(n).
5. **Когда использовать**
    - Когда нужен массив с динамическим размером и быстрый доступ по индексу — почти всегда `vector` лучший выбор.

Пример кода (работа с vector):

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3};
    std::cout << "size = " << v.size() << ", capacity = " << v.capacity() <<
"\n";

    // Добавим элементы и посмотрим на перевыделения
    for (int i = 4; i <= 20; ++i) {
        v.push_back(i);
        std::cout << "after push " << i << ": size=" << v.size()
                  << " capacity=" << v.capacity() << "\n";
    }

    // Резервирование
    std::vector<int> w;
    w.reserve(100);
    std::cout << "w capacity after reserve(100): " << w.capacity() << "\n";

    // Доступ к данным (C-совместимый указатель)
    int *data = v.data();
    std::cout << "First element via data(): " << data[0] << "\n";

    // Вставка в середину (дорого)
    v.insert(v.begin() + 2, 999);
    std::cout << "After insert: ";
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}
```

31. Контейнер `map`. назначение, инициализация, принцип работы

Пояснение (уровень — студент):
`std::map<Key, T>` — ассоциативный контейнер из стандартной библиотеки, который хранит пары ключ→значение (`std::pair<const Key, T>`). Основные свойства:

- Элементы упорядочены по ключу (по умолчанию — с помощью `operator<`), то есть `map` — это ассоциативный сбалансированный бинарный поиск (в реализации обычно красно-чёрное дерево).
- Ключи уникальны (в отличие от `multimap`).
- Доступ по ключу: `operator[]` (вставляет значение по умолчанию, если ключ отсутствует) и `at()` (кидает исключение при отсутствующем ключе).
- Вставка/удаление/поиск — логарифмическая сложность `O(log n)`.
- Итераторы двунаправленные, проходят элементы в порядке возрастания ключей.

Инициализация:

- Список инициализации: `std::map<std::string,int> m = { {"a",1}, {"b",2} };`
- Пустой и последующие `insert`, `emplace`, `operator[]`.
- Можно задать кастомный компаратор: `std::map<Key,T,Compare>`.

Когда выбирать `map`:

- Нужен упорядоченный словарь с быстрым логарифмическим поиском;
- Нужен проход в отсортированном порядке по ключам.

Пример кода (map):

```
#include <iostream>
#include <map>
#include <string>

int main() {
    // Инициализация списком
    std::map<std::string, int> ages = {
        {"Alice", 23},
        {"Bob", 30},
        {"Charlie", 27}
    };

    // Вставка/обновление
    ages["Diana"] = 22;
    ages.insert(std::pair<std::string, int>("Eve", 35));

    // Поиск
    std::map<std::string, int>::iterator it = ages.find("Bob");
    if (it != ages.end()) {
        std::cout << "Bob: " << it->second << '\n';
    }

    // Доступ с проверкой
    try {
        std::cout << "Alice: " << ages.at("Alice") << '\n';
```

```
    } catch (const std::out_of_range&) {
        std::cout << "No Alice\n";
    }

    // Проход по map
    for (const std::pair<const std::string, int>& p : ages) {
        std::cout << p.first << " => " << p.second << '\n';
    }

    // Кастомный компаратор (обратный порядок)
    std::map<int, std::string, std::greater<int>> rev;
    rev[1] = "one";
    rev[10] = "ten";

    std::cout << "Keys in descending order:\n";
    for (const std::pair<const int, std::string>& pr : rev)
        std::cout << pr.first << ' ' << pr.second << '\n';
}
```

32. Контейнер `set`. назначение, инициализация, принцип работы

Пояснение:
`std::set<T>` — контейнер уникальных элементов, упорядоченных по значению (по умолчанию `operator<`). Поведение очень похоже на `map`, но хранится только ключ (нет связанного значения). Свойства:

- Элементы уникальны.
- Упорядочен по компаратору.
- Операции вставки/удаления/поиска — `O(log n)`.
- Подходит, когда нужен набор уникальных элементов и быстрый поиск принадлежности.
- Есть `multiset` для допуска дубликатов.

Инициализация и методы: `insert`, `emplace`, `erase`, `find`, `count` (вернёт 0/1), `lower_bound`, `upper_bound`.

Пример кода (set):

```
#include <iostream>
#include <set>
#include <vector>

int main() {
    std::set<int> s = {5, 1, 3, 5}; // дубликат 5 проигнорируется
    s.insert(4);
    s.emplace(2);

    int x = 3;
    if (s.find(x) != s.end()) {
        std::cout << x << " есть в множестве\n";
    }

    std::cout << "Количество элементов: " << s.size() << '\n';

    for (int v : s) std::cout << v << ' ';
    std::cout << '\n';
```

```
    std::vector<int> v = {1, 2, 2, 3, 3, 3, 4};
    std::set<int> unique(v.begin(), v.end());

    std::cout << "Уникальные: ";
    for (int a : unique) std::cout << a << ' ';
    std::cout << '\n';
}
```

33. Особенности библиотеки `algorithm`. Лямбда-выражения

Пояснение (алгоритмы):
`<algorithm>` содержит набор обобщённых функций для работы с диапазонами (итераторами): `sort`, `find`, `for_each`, `transform`, `accumulate` (в `<numeric>`), `copy`, `remove_if`, `partition`, `lower_bound`/`upper_bound`, `binary_search` и др. Они принимают итераторы начала/конца диапазона и часто — пользовательские функции или функторы для сравнения/обработки. Ключевые идеи:

- Алгоритмы отделены от контейнеров — работают через итераторы.
- Можно комбинировать алгоритмы для выразительного и компактного кода.
- Многие алгоритмы требуют определённого типа итератора (напр., `sort` — random access).

Пояснение (лямбды):
Лямбда-выражение — это анонимная функция прямо в месте использования. Общая форма:
`[capture](parameters) -> return_type { body }` — часто `-> return_type` опускают. `capture` определяет какие внешние переменные захватываются: `[]` (ничего), `[=]` (по значению), `[&]` (по ссылке), `[a,&b]` (микширование). Лямбды широко используются с `<algorithm>` для компактных пользовательских предикатов и операций.

Пример кода (algorithm + lambda):

```
#include <algorithm>
#include <iostream>
#include <vector>
#include <numeric> // для iota и accumulate

int main() {
    std::vector<int> v(10);
    std::iota(v.begin(), v.end(), 1); // v = {1,2,...,10}

    v.erase(std::remove_if(v.begin(), v.end(), [](int x){ return x % 2 == 0;
}), v.end());

    std::cout << "После удаления чётных: ";
    for (int x : v) std::cout << x << ' ';
    std::cout << '\n';

    int sum = std::accumulate(v.begin(), v.end(), 0, [](int acc, int x){
return acc + x; });
    std::cout << "Сумма: " << sum << '\n';

    std::vector<int> a = {5, 1, 8, 3, 2};
    std::sort(a.begin(), a.end(), [](int lhs, int rhs){
```

```
        return lhs > rhs;
    });

    std::cout << "Отсортировано по убыванию: ";
    for (int x : a) std::cout << x << ' ';
    std::cout << '\n';
}
```

34. Составные типы данных в C++ (`enum`, `union`, `typedef`, битовые поля)

Пояснение:

- `enum` — перечисление. Есть классические `enum` и `enum class` (scoped). `enum class` безопаснее: нет неявных преобразований в целое, имена не «утекают» в область видимости. Пример: `enum class Color { Red, Green, Blue };`.
- `union` — объединение: несколько полей разделяют одну область памяти. Полезно для экономии памяти и low-level задач. **Важно:** хранится только одно активное поле; чтение неинициализированного поля — неопределено (за исключением некоторых трюков с `std::variant`). В C++ безопаснее использовать `std::variant`.
- `typedef` / `using` — псевдонимы типов. `using` — предпочтительнее в современном C++: `using str_t = std::string;`.
- Битовые поля (bit-fields) — позволяют явно указать размер поля в битах внутри `struct`/`union`, например `unsigned flags:1;`. Полезно для работы с битовыми флагами и экономии памяти, но поведение зависит от порядка битов и выравнивания — не использовать для межплатформенного сетевого формата без явной сериализации.

Пример кода (enum, union, typedef/using, битовые поля):

```
#include <iostream>
#include <variant>
#include <cstring>
#include <string>

enum class Color : uint8_t { Red = 1, Green = 2, Blue = 3 };

union IntOrFloat {
    int i;
    float f;
    IntOrFloat() { i = 0; }
};

using string_t = std::string;

struct Flags {
    unsigned is_valid : 1;
    unsigned error_code : 3;
    unsigned reserved : 4;
};

int main() {
    Color c = Color::Green;
    if (c == Color::Green) std::cout << "Green selected\n";

    IntOrFloat u;
    u.f = 3.14f;
```

```
    int asInt;
    std::memcpy(&asInt, &u.f, sizeof(asInt));
    std::cout << "Float bits as int: " << asInt << '\n';

    string_t s = "Hello";

    Flags ff{};
    ff.is_valid = 1;
    ff.error_code = 5;
    std::cout << "Flags: is_valid=" << ff.is_valid << " err=" <<
ff.error_code << '\n';

    std::variant<int, double, std::string> v;
    v = 10;
    v = std::string{"text"};
}
```

35. Структура: понятие, особенности использования. Инициализация структур. Доступ к полям структуры

Пояснение:
`struct` в C++ — агрегатный тип (как `class` с публичным доступом по умолчанию). Используется для объединения данных; также может содержать функции-члены, конструкторы, приватные поля и т.д. Отличия `struct`/`class` только в default access и default inheritance (`struct` — public, `class` — private). Пара моментов:

- Инициализация: агрегатная инициализация `{...}`, списочная инициализация, конструкторы.
- Доступ к полям: `obj.field` или `ptr->field`.
- Можно перегружать операторы, добавлять методы, конструкторы/деструкторы.
- Для POD/агрегатов — удобны для бинарной сериализации (с осторожностью).

Пример инициализации:

- `S s{a,b};` — aggregate initialization.
- `S s = { ... };`

Пример кода (struct):

```
#include <iostream>
#include <string>

struct Person {
    std::string name;
    int age;

    void print() const {
        std::cout << name << " (" << age << ")\n";
    }

    Person(std::string n, int a) : name(std::move(n)), age(a) {}
};

int main() {
    Person p1("Ivan", 21);
    p1.print();
```

```
    struct Point { double x; double y; };
    Point pt = {1.0, 2.0};
    std::cout << "Point: " << pt.x << ", " << pt.y << '\n';

    Person* pp = &p1;
    pp->age = 22;
    pp->print();
}
```

36. Файловые потоки

Пояснение:
Файловые потоки в стандартной библиотеке: `<fstream>` обеспечивает `std::ifstream` (чтение), `std::ofstream` (запись), `std::fstream` (чтение/запись). Работа в текстовом или бинарном режиме. Основы:

- Открытие: `std::ifstream in("file.txt");` или `in.open("file.txt");`
- Проверка состояния: `if (!in)` или `if (in.fail())`.
- Чтение строк: `std::getline(in, line)`.
- Чтение/запись бинарных данных: `read(reinterpret_cast<char*>(&obj), sizeof(obj))`.
- Всегда закрывать файл (`close()`), хотя деструктор делает это автоматически.
- Режимы открытия: `std::ios::in`, `std::ios::out`, `std::ios::binary`, `std::ios::app`, `std::ios::trunc`.

Важно: при бинарной сериализации для переносимости нужно заботиться о выравнивании, порядке байтов и версиях структуры.

Пример кода (файловые потоки):

```
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

int main() {
    {
        std::ofstream out("sample.txt");
        if (!out) { std::cerr << "Не могу открыть файл для записи\n"; return
1; }
        out << "Hello\n";
        out << 123 << '\n';
    }

    {
        std::ifstream in("sample.txt");
        std::string line;
        while (std::getline(in, line)) {
            std::cout << "Line: " << line << '\n';
        }
    }

    std::vector<int> data = {1, 2, 3, 4, 5};
    {
        std::ofstream bout("data.bin", std::ios::binary);
```

```
        size_t n = data.size();
        bout.write(reinterpret_cast<const char*>(&n), sizeof(n));
        bout.write(reinterpret_cast<const char*>(data.data()), n *
sizeof(int));
    }

    {
        std::ifstream bin("data.bin", std::ios::binary);
        size_t n;
        bin.read(reinterpret_cast<char*>(&n), sizeof(n));
        std::vector<int> readData(n);
        bin.read(reinterpret_cast<char*>(readData.data()), n * sizeof(int));

        std::cout << "Read binary: ";
        for (int x : readData) std::cout << x << ' ';
        std::cout << '\n';
    }
}
```

37. Перегрузка операций. Пример реализаций для структуры и для контейнеров

Пояснение:
Перегрузка операторов позволяет определить поведение операторов (`+`, `-`, `[]`, `<<`, `==`, `!=`, `<` и т.д.) для пользовательских типов. Рекомендации:

- Часто перегружают `operator<<` для печати объектов, `operator==/!=` для сравнения, `operator<` для использования в `std::set`/`std::map`.
- Перегрузки лучше делать в виде свободных функций (некоторые операторы) или методов (например, `operator[]` для контейнера).
- Не перегружать операторы в неожиданных смыслах.

Для структуры: реализуем `operator+`, `operator<<`, `operator<`.

Для контейнеров: нельзя изменить стандартные контейнеры, но можно определить шаблонный `operator<<` для печати стандартных контейнеров (свободная функция). Также можно написать свою простую обёртку-контейнер с `operator[]`.

Пример кода (перегрузка операторов):

```
#include <iostream>
#include <vector>
#include <map>
#include <string>

// Пример структуры с перегрузками
struct Vec2 {
    double x, y;

    Vec2 operator+(const Vec2& other) const {
        return Vec2{x + other.x, y + other.y};
    }

    bool operator<(const Vec2& o) const {
        if (x != o.x) return x < o.x;
        return y < o.y;
```

```
    }

    bool operator==(const Vec2& o) const {
        return x == o.x && y == o.y;
    }
};

std::ostream& operator<<(std::ostream& os, const Vec2& v) {
    os << '(' << v.x << ',' << v.y << ')';
    return os;
}

template<typename T>
std::ostream& operator<<(std::ostream& os, const std::vector<T>& vec) {
    os << '[';
    bool first = true;
    for (typename std::vector<T>::const_iterator it = vec.begin(); it !=
vec.end(); ++it) {
        if (!first) os << ", ";
        first = false;
        os << *it;
    }
    os << ']';
    return os;
}

int main() {
    Vec2 a{1, 2}, b{3, 4};
    Vec2 c = a + b;
    std::cout << "c = " << c << '\n';

    std::vector<Vec2> arr = {a, b, c};
    std::cout << "vec = " << arr << '\n';

    std::map<Vec2, std::string> m;
    m[a] = "A";
    m[b] = "B";

    for (std::map<Vec2, std::string>::const_iterator it = m.begin(); it !=
m.end(); ++it)
        std::cout << it->first << " -> " << it->second << '\n';
}
```

38. Итераторы: понятие, операции. Арифметика итераторов. Примеры

Пояснение:
Итератор — абстракция указателя для доступа к элементам контейнера и прохода по диапазону. Категории итераторов:

1. **Input iterator** — однонаправленный чтение (например, `istream_iterator`).
2. **Output iterator** — однонаправленная запись (например, `back_inserter`).
3. **Forward iterator** — может проходить вперёд несколько раз.
4. **Bidirectional iterator** — можно двигаться вперёд и назад (`++`, `--`) — например, `std::list`, `std::set`, `std::map`.
5. **Random access iterator** — поддерживает `+`, `-`, `+=`, `-=` и сравнения `<`, `>`. Примеры: `std::vector`, `std::deque`, массивы.

Операции:

- Доступ к элементу: `*it`, `it->member`.
- Переход: `++it`, `--it` (если поддерживается).
- Сравнение: `it == it2`, `it != it2`.
- Для random access: `it + n`, `it - n`, `it[n]`, `it2 - it1` возвращает расстояние.

Арифметика итераторов:
Работает ТОЛЬКО для random access итераторов. Для bidirectional — только `++`/`--`.
Попытка применить `it + n` к `std::list::iterator` не скомпилируется.

Примеры использования:

- Перебор контейнера от `begin()` до `end()`.
- Использование алгоритмов требует итераторов определённой категории.

Пример кода (итераторы и арифметика):

```
#include <iostream>
#include <vector>
#include <list>
#include <algorithm>
#include <iterator>

int main() {
    std::vector<int> v = {10, 20, 30, 40, 50};

    std::vector<int>::iterator it = v.begin();
    it += 2; // теперь указывает на 30
    std::cout << "*it = " << *it << '\n';

    std::cout << "v[1] via iterator: " << *(v.begin() + 1) << '\n';

    std::ptrdiff_t d = v.end() - v.begin();
    std::cout << "size via iter diff = " << d << '\n';

    std::list<int> lst = {1, 2, 3, 4, 5};
    std::list<int>::iterator lit = lst.begin();
    ++lit;
    std::advance(lit, 2);
    std::cout << "list element after advance: " << *lit << '\n';

    std::vector<int> dest;
    std::back_insert_iterator<std::vector<int>> inserter(dest);
    std::copy(v.begin(), v.end(), inserter);

    std::cout << "Copied: ";
    for (int x : dest) std::cout << x << ' ';
    std::cout << '\n';

    std::ptrdiff_t distance_v = std::distance(v.begin(), v.end());
    std::cout << "distance v: " << distance_v << '\n';
}
```

39. DLL — понятие, концепция, плюсы/минусы, типы связывания, процесс создания, управление предупреждениями в g++, оптимизация, пути поиска и стандарты в g++, санитайзеры, отладка и gdb, отладочная информация

1) Что такое DLL (и .so)

Пояснение:
DLL (Dynamic Link Library) — динамически подключаемая библиотека (термин из мира Windows). Аналог в Unix/Linux — shared object (`.so`). Это бинарный модуль, содержащий код и данные, которые загружаются в адресное пространство процесса либо при запуске (implicit/implicit load), либо по запросу во время выполнения (explicit load). Цель — разделение кода на переиспользуемые модули, экономия памяти и возможность обновления библиотеки без перекомпиляции зависимых приложений.

Основные концепты:

- Экспорт/импорт символов (функций, переменных).
- Разрешение символов: на этапе загрузки или позднее (динамическое связывание).
- Версионирование ABI и управление несовместимостями.
- Положения о видимости символов (hidden/visible).

2) Плюсы (5) и минусы (2)

Плюсы:

1. Экономия памяти: несколько процессов могут совместно использовать секции кода библиотеки (код в памяти один).
2. Меньший размер исполняемого файла: код вынесен в библиотеки.
3. Обновляемость: можно обновить библиотеку без перекомпиляции или перезагрузки основного приложения (в идеале).
4. Плагинная архитектура: динамическая загрузка модулей (LoadLibrary/`dlopen`) позволяет строить расширяемые приложения.
5. Разделение команд разработки: библиотеки можно разрабатывать и тестировать отдельно.

Минусы:

1. Проблемы с совместимостью ABI: обновление библиотеки может сломать приложения (DLL hell).
2. Накладные расходы: динамическая загрузка/поиск/разрешение символов и возможные проблемы с производительностью при старте или при первом вызове.

3) Типы связывания (таблица по 5 критериям)

Критерии: момент связывания, способ разрешения символов, размер исполняемого, возможность замены/обновления, простота разработки.

Тип связывания	Момент связывания	Разрешение символов	Размер exe	Обновляемость	Простота
Статическое (static .a)	Время компоновки	Линкер связывает в бинарник	большой	нет (нужна рекомпиляция)	просто
Динамическое (implicit .so/.dll)	При запуске	Загрузчик OS разрешает	меньше	можно заменить (с осторожностью)	обычно

Тип связывания	Момент связывания	Разрешение символов	Размер exe	Обновляемость	Простота
Явное/динамическое (dlopen/LoadLibrary)	Во время выполнения	Разрешение при `dlopen`/`GetProcAddress`	меньше	высокая гибкость	сложнее (руч. управление)
Отложенная загрузка (lazy)	При первом обращении	Разрешение по требованию	меньше	замена возможна	требует поддержки загрузчика
Плагины (определённый API)	Во время выполнения	Символы через явный интерфейс	зависит	хорошая (изолированная)	требует проектирования API

4) Процесс создания DLL / shared library

Linux (g++) — .so:

- Компиляция с позиционно-независимым кодом: `-fPIC`
- Линковка в shared object: `-shared -fPIC -o libmylib.so mylib.o`
  Пример:

```
g++ -std=c++17 -O2 -fPIC -c mylib.cpp -o mylib.o
g++ -shared -o libmylib.so mylib.o
g++ -std=c++17 -o app main.cpp -L. -lmylib
LD_LIBRARY_PATH=. ./app
```

Windows (MSVC/g++-mingw) — .dll:

- Экспорт символов: `__declspec(dllexport)` / `__declspec(dllimport)` или использовать `.def`.
- Команда (MinGW): `g++ -shared -o mylib.dll mylib.cpp`

Я приведу пример создания .so и использования его в C++ ниже.

5) Управление предупреждениями в g++

Ключи:

- `-Wall` — большинство полезных предупреждений.
- `-Wextra` — дополнительные предупреждения.
- `-Werror` — превращает предупреждения в ошибки.
- `-Wno-...` — выключить конкретное предупреждение.
- `-pedantic`/`-pedantic-errors` — строгая проверка соответствия стандарту.
- Можно локально подавлять предупреждения внутри кода:
- `#pragma GCC diagnostic push`
- `#pragma GCC diagnostic ignored "-Wunused-parameter"`
- `// код`
- `#pragma GCC diagnostic pop`

6) Оптимизация в g++

Уровни оптимизации:

- `-O0` — никакой оптимизации (по умолчанию в отладке).
- `-O1`, `-O2` — безопасные оптимизации.
- `-O3` — более агрессивно.
- `-Ofast` — включает небезопасные оптимизации (нарушение строго стандарта).
- `-Os` — оптимизация на размер.
  Дополнительно:
- `-flto` — link time optimization (LTO).
- `-march=native`, `-mtune=...` — использование инструкций CPU.
- Комбинация `-g -O2` — включает оптимизацию + дебагинфо (хотя оптимизации усложняют отладку).

7) Пути для поиска и стандарты языка в g++

Include и библиотечные пути:

- `-I/path/to/includes` — добавить директиву поиска заголовков.
- `-L/path/to/libs` — путь для линковщика (добавить либы).
- `-Wl,-rpath,/path` или `-Wl,-rpath='$ORIGIN'` — runtime rpath.
- `LD_LIBRARY_PATH` — временный путь поиска `.so` в рантайме (не рекомендуется в продакшне).

Стандарты:

- `-std=c++98`, `-std=c++11`, `-std=c++14`, `-std=c++17`, `-std=c++20` и т.д.
- `-std=gnu++17` включает GNU-расширения.

8) Санитайзеры (sanitizers) — понятие, виды и механизм

Пояснение:
Санитайзеры — это инструменты динамического анализа, встроенные в компилятор/рантайм (LLVM/Clang, GCC поддержка), которые автоматически вставляют дополнительную проверку исполнения для нахождения классов ошибок: переполнения, use-after-free, data race и т.д.

Основные виды:

- AddressSanitizer (ASan): обнаруживает буферные переполнения, use-after-free, use-after-return (частично).
- LeakSanitizer (LSan): поиск утечек памяти (часто интегрирован в ASan).
- UndefinedBehaviorSanitizer (UBSan): ловит неопределённое поведение (деление на 0, некорректные приведения).
- ThreadSanitizer (TSan): обнаруживает гонки данных в многопоточном коде.
- MemorySanitizer (MSan): детектирует чтение неинициализированной памяти (требует компиляции всего приложения и зависимостей с MSan).

Механизм: компилятор вставляет «инструментирующие» проверки в байткод/инструкции; при срабатывании — рантайм печатает детальную трассировку и (часто) дамп стека. Используются специальные рантайм-библиотеки.

Пример использования:

```
g++ -std=c++17 -fsanitize=address,undefined -g -O1 myprog.cpp -o myprog
./myprog
```

ASan и UBSan при обнаружении ошибки покажут стек, источник (файл:строка) и подсказки.

9) Понятие отладки, gdb и возможности

Отладка: процесс пошагового изучения выполнения программы, поиска дефектов (багов) с помощью точек останова, инспекции переменных, трассировок, анализа дампов памяти.

gdb (GNU Debugger): мощный текстовый отладчик. Возможности:

- Установка точек останова (`break`), условных точек.
- Пошаговое выполнение: `step` (заходить в функции), `next` (шаг без захода).
- Продолжение выполнения: `continue`.
- Просмотр стека: `backtrace`.
- Просмотр и изменение переменных: `print var`, `set var=...`.
- Просмотр регистров, ассемблерного кода: `info registers`, `disassemble`.
- Watchpoints — следить за изменением памяти.
- Отладка core-файлов: `gdb prog core`.
- Remote debugging: gdbserver/gdb.

10) Отладочная информация в g++

- `-g` — генерировать отладочные символы (DWARF). Самый распространённый формат.
- `-g3` — включает макроинформацию.
- `-ggdb` — генерировать отладочную информацию, специфичную для gdb.
- `-gsplit-dwarf` / `-fdebug-prefix-map` — уменьшение размера исполняемого путем вынесения debug info.
- Отладочная информация позволяет gdb сопоставлять машинный код и исходники, печатать значения переменных, имена типов, и т. п.

Пример: простая shared library + приложение + сборка + запуск санитайзера и gdb

Файлы: `mylib.cpp`, `mylib.hpp`, `main.cpp`.

mylib.hpp

```
#ifndef MYLIB_HPP
#define MYLIB_HPP

extern "C" {

// Экспортируемая C-совместимая функция
int add_ints(int a, int b);

}

#endif // MYLIB_HPP
```

mylib.cpp

```
#include "mylib.hpp"

int add_ints(int a, int b) {
    return a + b;
}
```

main.cpp

```
#include <iostream>
#include "mylib.hpp"

int main() {
    int x = 10;
    int y = 20;
    int z = add_ints(x, y);
    std::cout << "sum = " << z << std::endl;
    return 0;
}
```

Сборка и запуск (в комментариях):

```
# Компиляция shared library с отладочной информацией
g++ -std=c++17 -fPIC -g -O2 -c mylib.cpp -o mylib.o
g++ -shared -o libmylib.so mylib.o

# Компиляция приложения со ссылкой на библиотеку
g++ -std=c++17 -g -O2 -L. -o app main.cpp -lmylib

# В рантайме:
LD_LIBRARY_PATH=. ./app

# Запуск с AddressSanitizer (компилировать с -fsanitize)
g++ -std=c++17 -fPIC -fsanitize=address,undefined -g -O1 -c mylib.cpp -o
mylib.o
g++ -shared -o libmylib.so mylib.o
g++ -std=c++17 -fsanitize=address,undefined -g -O1 -L. -o app main.cpp -
lmylib
LD_LIBRARY_PATH=. ./app

# Отладка:
gdb ./app
# В gdb: (пример)
# break main
# run
# step / next / print z
```

---

40. Страничная память, виртуальный адрес, страница (Page), реактивное и функциональное программирование, разрядность регистров EAX и др.

Страничная память, виртуальный адрес, страница

Пояснение:

- **Виртуальный адрес (VA)** — адрес, который использует программа; это представление адресного пространства процесса. Процесс видит непрерывную память, но физическая память (RAM) отображается через таблицы страниц (MMU).

- **Страница (Page)** — минимальный блок управления памятью в виртуальной памяти; обычно размер 4 KiB (может быть 2 MiB, 1 GiB — huge pages).
- **Страничная память (paging)** — механизм, при котором виртуальные страницы сопоставляются с физическими фреймами; при обращении к странице, не имеющей физического фрейма, возникает page fault и ОС либо загружает страницу из swap/file, либо выдаёт ошибку (segfault).
- Таблицы страниц (page tables) хранят соответствия VA -> PA (физический адрес), бит присутствия, права доступа (R/W/X), бит доступа (dirty/accessed).

Практический смысл: программист может использовать `mmap`, `mlock`, `mprotect` для управления страницами.

Пример: узнать размер страницы и выделить память, выровненную по странице

```
#include <unistd.h>
#include <iostream>
#include <cstdlib>

int main() {
    long page_size = sysconf(_SC_PAGESIZE);
    if (page_size == -1) {
        std::cerr << "sysconf failed\n";
        return 1;
    }
    std::cout << "Page size = " << page_size << " bytes\n";

    void* ptr = nullptr;
    int res = posix_memalign(&ptr, static_cast<size_t>(page_size),
static_cast<size_t>(page_size));
    if (res != 0) {
        std::cerr << "posix_memalign failed\n";
        return 1;
    }
    std::cout << "Allocated page-aligned block at " << ptr << '\n';
    std::free(ptr);
    return 0;
}
```

Реактивное программирование (Reactive programming)

Пояснение:
Реактивное программирование — парадигма, ориентированная на потоки данных и распространение изменений: вы описываете потоки событий и зависимости между ними, а система автоматически распространяет изменения (push-based). Часто используются observable/observer, backpressure, асинхронные стримы. Примеры: RxJava, ReactiveX, Reactor. Полезно для UI, работы с потоками событий и асинхронного ввода-вывода.

Функциональное программирование (Functional programming)

Пояснение:
Парадигма, где функции — первоклассные сущности, предпочтение чистым функциям (без побочных эффектов), неизменяемость данных, выражения и композиция функций, рекурсия. В C++ элементы функционального стиля возможны: `std::function`, лямбды, алгоритмы `<algorithm>`, `std::transform`, `std::accumulate`. ФП облегчает рассуждения о коде и тестирование.

Общая разрядность регистров (EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP)

Пояснение:

- `EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP` — это 32-битные регистры x86 (IA-32). Их имена начинаются с `E` — extended (расширенные 16-битных AX/BX/...).
- В x86-64 регистры называются `RAX, RBX, RCX, RDX, RSI, RDI, RSP, RBP` и имеют 64 бита; младшие 32 бита совпадают с `EAX` и т.д. Низшие 16 бит: `AX`, младшие 8: `AL/AH`.
- Резюме: в IA-32 — 32-битный размер; в x86-64 — 64-битный.

41. Адресное пространство процесса, изоляция процессов

Пояснение:

- Каждому процессу ОС предоставляет собственное виртуальное адресное пространство — видимую непрерывную область адресов, обычно разделённую на сегменты/лайды: текст (код), данные, heap, stack, mmap-области.
- **Изоляция процессов** означает, что адреса одного процесса не видимы (и не доступны) другому процессу напрямую: попытка доступа приведёт к page fault и сигналу `SIGSEGV`. Это основа безопасности и стабильности — баг в одном процессе не может повредить память другого.
- Общение между процессами осуществляется через IPC (pipes, sockets, shared memory, files).
- Права доступа (пользователь/группа) и механизмы SELinux/AppArmor/namespace/containers добавляют дополнительные уровни изоляции.

Демонстрация: fork и изоляция памяти (child меняет переменную — родитель не видит)

```
#include <iostream>
#include <sys/types.h>
#include <unistd.h>
#include <sys/wait.h>

int global_var = 42;

int main() {
    pid_t pid = fork();
    if (pid == -1) {
        std::cerr << "fork failed\n";
        return 1;
    } else if (pid == 0) {
        // child
        global_var = 100;
        std::cout << "Child: global_var = " << global_var << std::endl;
        return 0;
    } else {
        // parent
        int status = 0;
        waitpid(pid, &status, 0);
        std::cout << "Parent: global_var = " << global_var << std::endl;
    }
    return 0;
}
```

Вывод покажет, что parent всё ещё видит 42 — потому что fork создал копию адресного пространства (copy-on-write).

42. Основы работы в Linux

Пояснение (ключевые вещи для начинающего):

1. **Файловая система:** всё — файл. Точки монтирования, права доступа (rwx), владельцы и группы.
2. **Процессы:** PID, PPID, fork/exec, сигналы (`kill`), планирование, `ps`, `top`.
3. **Права:** root vs обычный пользователь; `sudo`.
4. **Пакетный менеджмент:** apt/yum/pacman/другие дистрибутивные менеджеры.
5. **Службы (systemd):** `systemctl start/stop/status`.
6. **Права доступа к файлам:** `chmod`, `chown`.
7. **Редиректы и конвейеры:** `>`, `>>`, `|`.
8. **Утилиты:** `grep`, `awk`, `sed`, `strace`, `ltrace`, `ldd`, `ldconfig`.
9. **Развитая система прав доступа и сетевых настройок.**
10. **Отладка и профилирование:** `gdb`, `valgrind`, `perf`.

Полезные практические команды (коротко)

- `ls -l`, `stat file`
- `ps aux | grep name`, `top`/`htop`
- `strace ./app` — системные вызовы
- `ldd ./app` — какие shared libs нужны
- `nm libmylib.so` — символы в библиотеке
- `file ./app` — тип бинарника (ELF, архитектура)
- `man` — документация